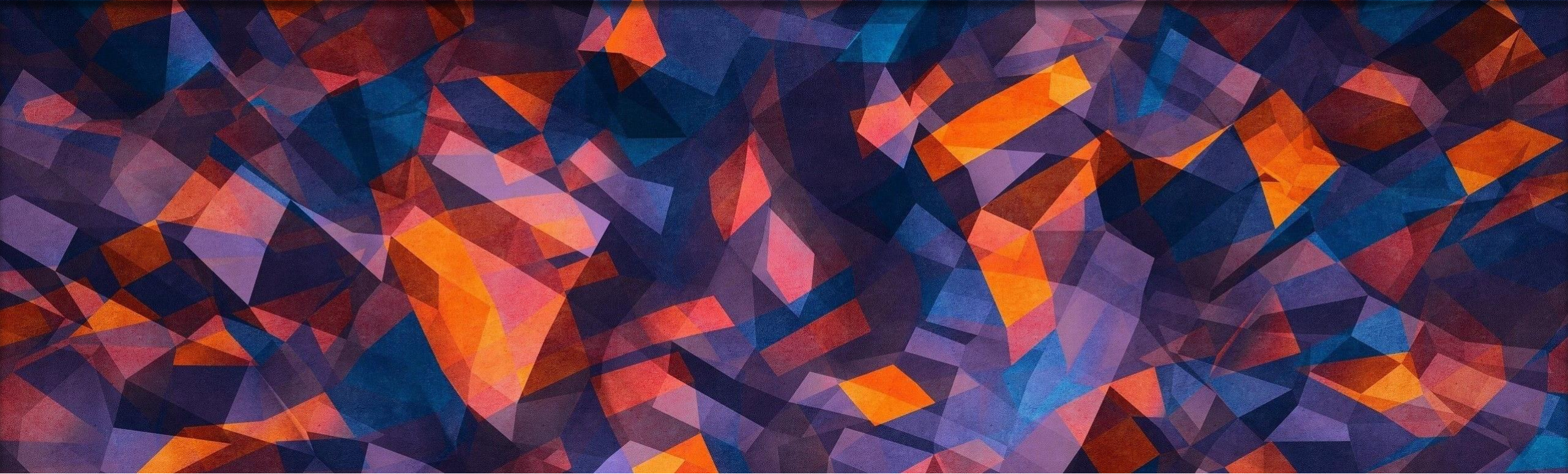


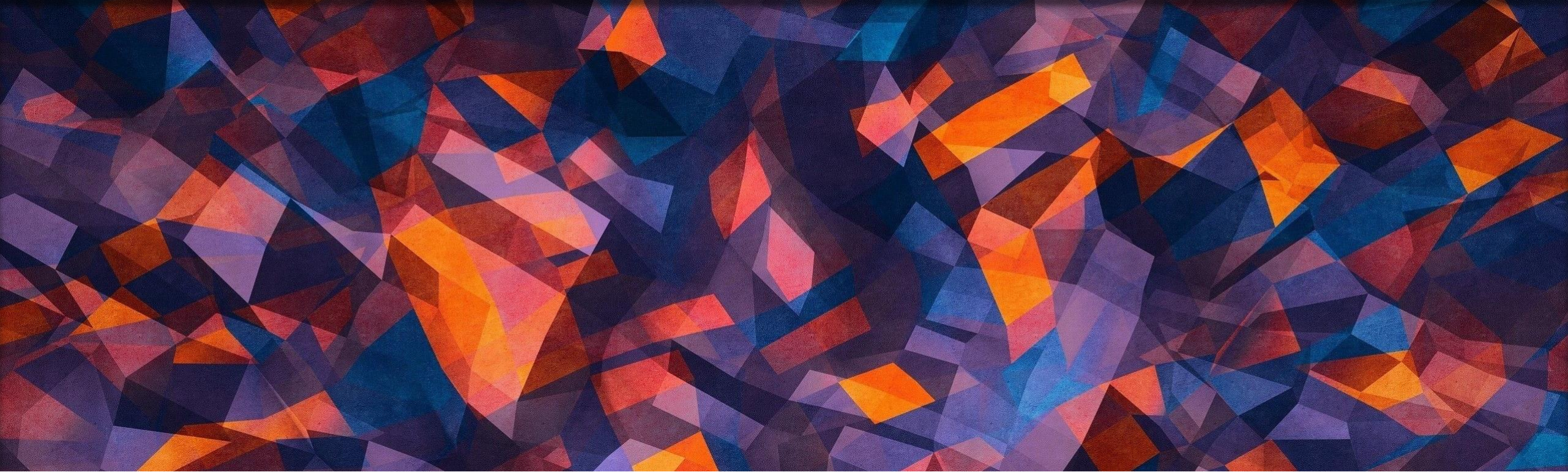


Event-driven and Process-oriented Architectures

Stream Processing – Windows and Time

Lecture 10

- Relationship of events and time
- Introduction into windows
- Kafka Streams Processor API
- The Event Streaming Database ksqlDB



Event-driven and Process-oriented Architectures

Relationship of Events and Time

Lecture 10

Children's Healthcare of Atlanta uses Kafka Streams and ksqlDB to make real-time predictions about whether or not children with head trauma will need surgical intervention in the near future.

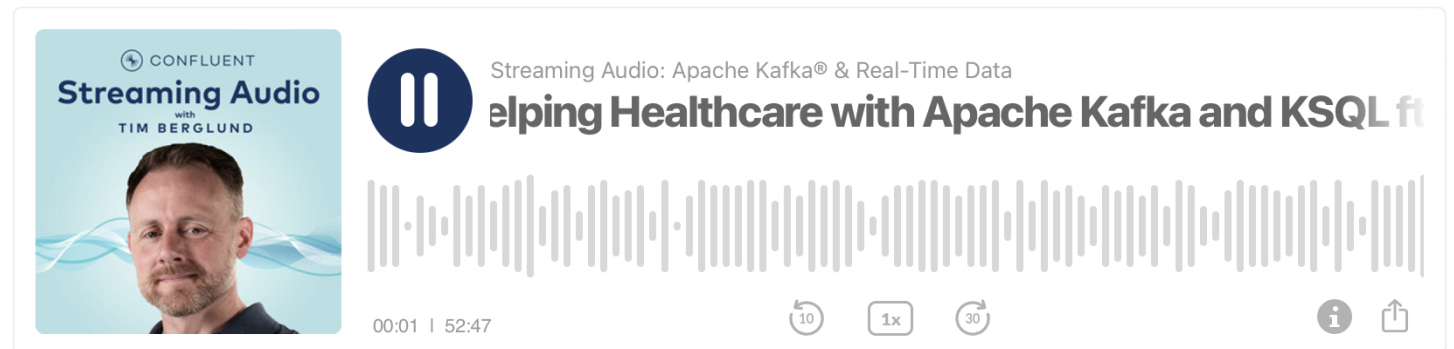
How:

- measuring intracranial pressure, aggregating the measurements, and sending the aggregations to a predictive model.
- When the model predicts that the pressure will reach dangerous levels in the next 30 minutes, then healthcare professionals will be notified so they can take the appropriate action.

Streaming Audio: Apache Kafka® & Real-Time Data

Helping Healthcare with Apache Kafka and KSQL ft. Ramesh Sringeri

AUGUST 28, 2019 CONFLUENT, ORIGINAL CREATORS OF APACHE KAFKA® SEASON 1 EPISODE 50



LISTEN ON



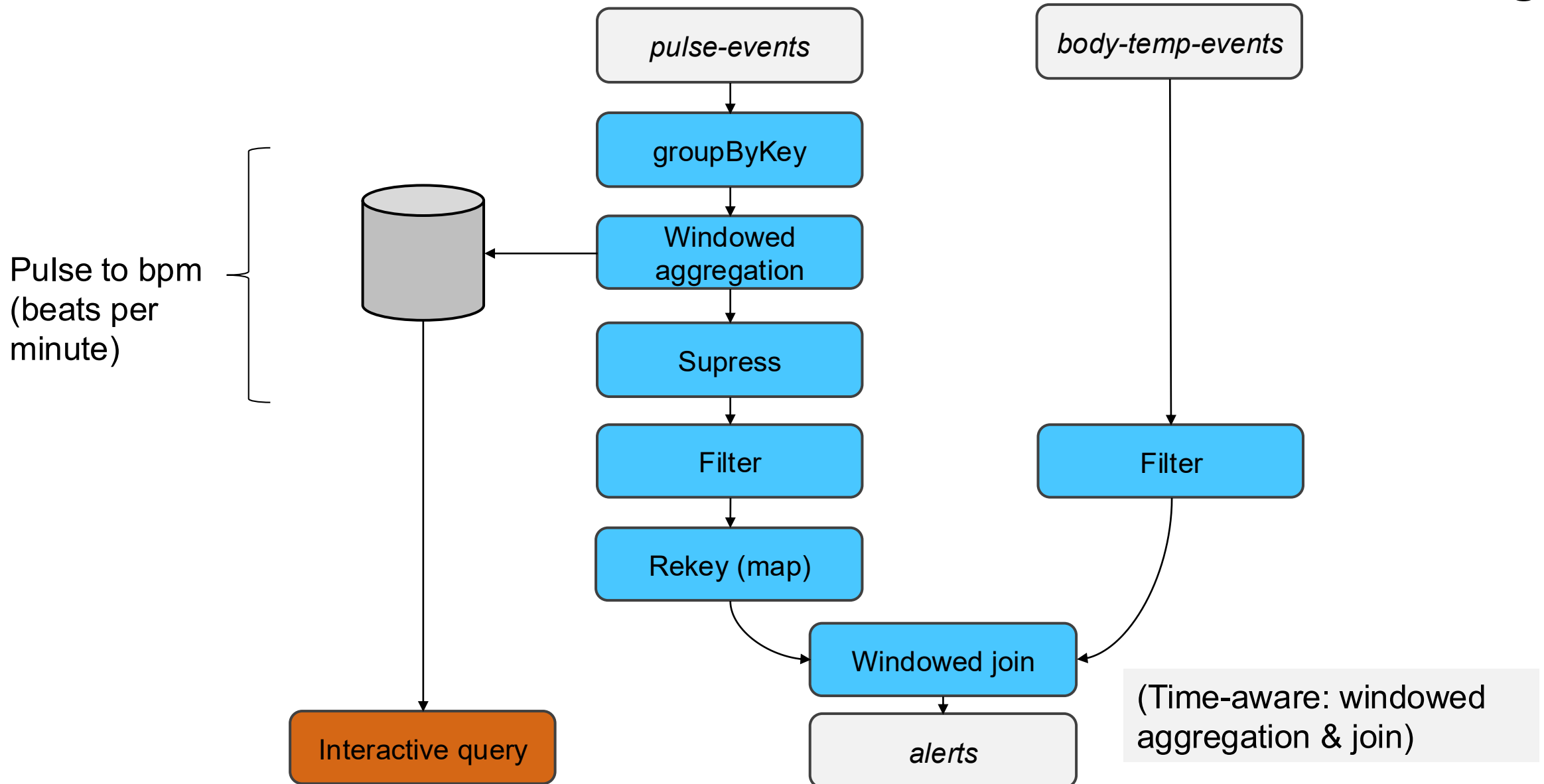
SHARE EPISODE



[Listen to interview](#) of Tim Berglung's interview with Ramesh Sringeri concerning this use case and how time-aware stream processing makes all of this possible

Example Topology

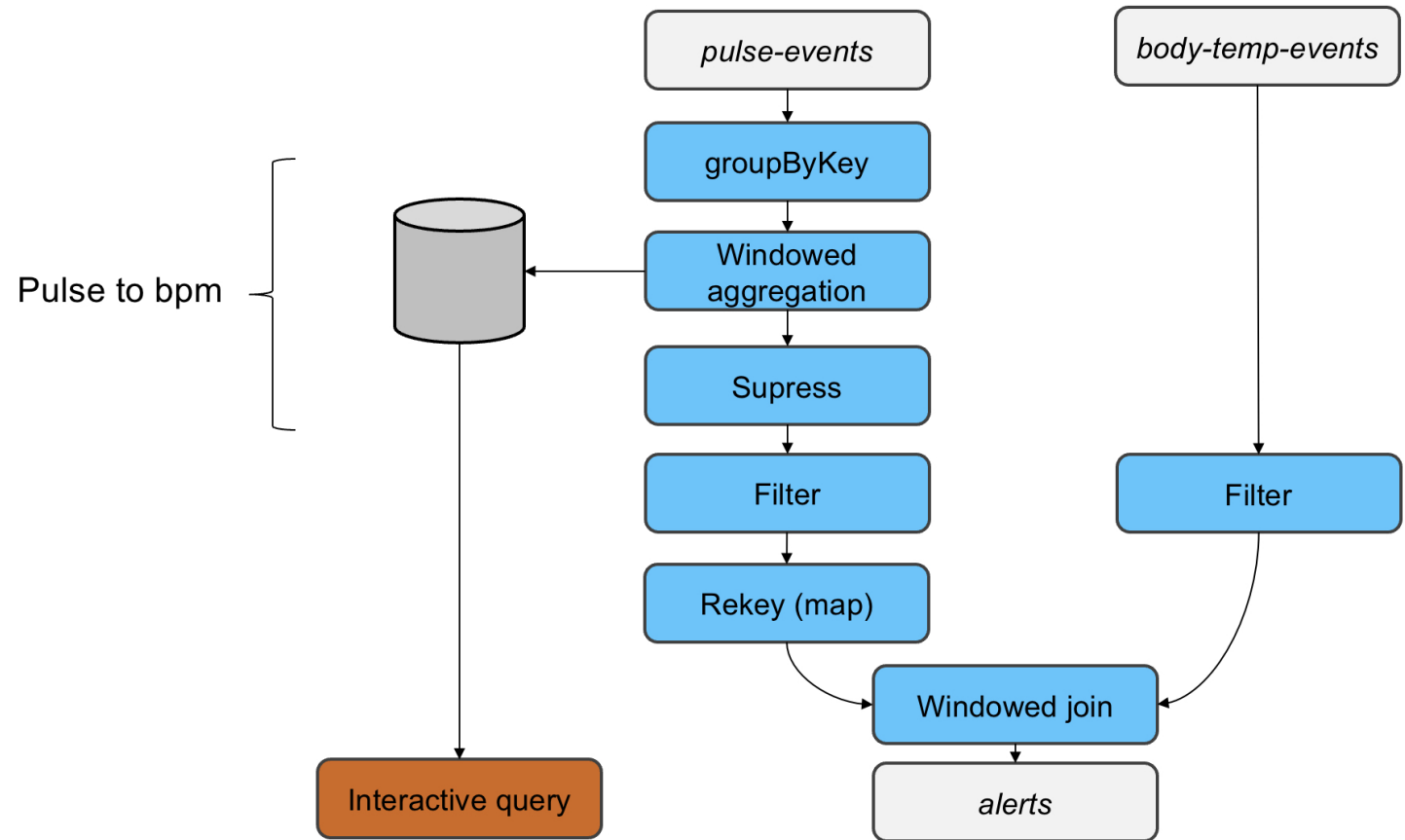
Time-aware Stream Processing



Example Topology

Time-aware Stream Processing

- Detect the presence of a medical condition called *systemic inflammatory response syndrome*, or SIRS
- Several vital signs, including body temperature, blood pressure, and heart rate, that can be used as indicators of SIRS.
- Tutorial covers two of these measurements: body temperature and heart rate. When both of these vitals reach predefined thresholds, a record is sent to an *alerts* topic to notify the appropriate medical personnel.



Example Topology

Time-aware Stream Processing

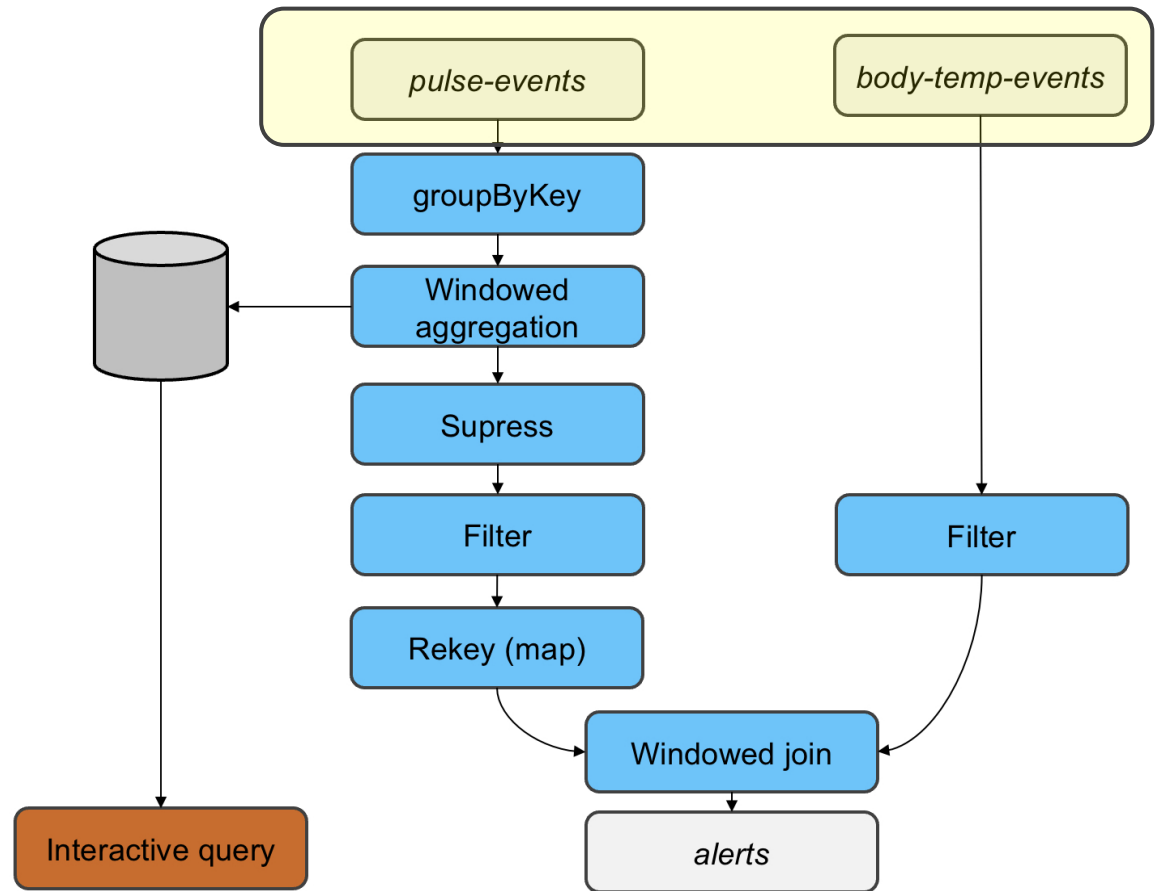
```
{  
  "timestamp": "2020-11-05T09:02:00.000Z"  
}
```

Example pulse-event
(timestamp = event time)

```
{  
  "timestamp": "2020-11-04T09:02:06.500Z",  
  "temperature": 101.2,  
  "unit": "F"  
}
```

Example body-temp-event

Pulse to bpm



When performing time-aware operations (windowed joins and aggregations), choosing the correct time semantics is important

- **Event time:** Time **when** the event actually **occurred** (at the data source); timestamp can be embedded in the payload or set directly by the Kafka producer
- **Log append time / Ingestion time:** The time **when** the event **arrived** at the Kafka broker and is appended to a topic, i.e., the ingestion time
- **Processing time:** The time **when** the stream processing application **processed** the event; reprocessing the same data (e.g., for bug fixes) will lead to new processing timestamps

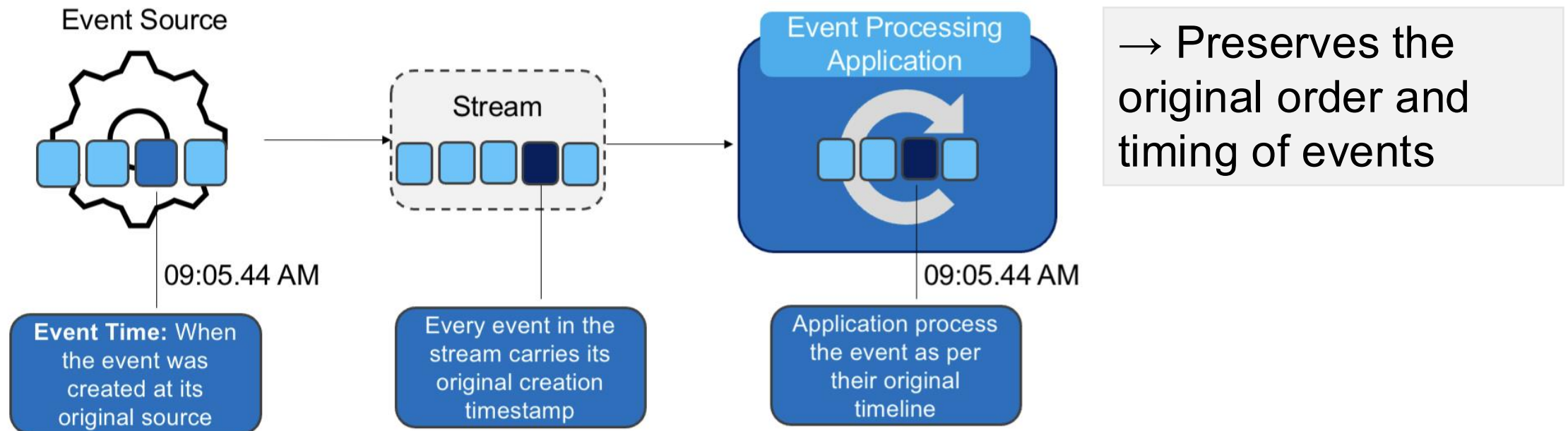
Why time matters:

Time semantics determine correctness of windows, joins, and aggregations

→ In practice: event time is preferred for correctness

Event-time Processing

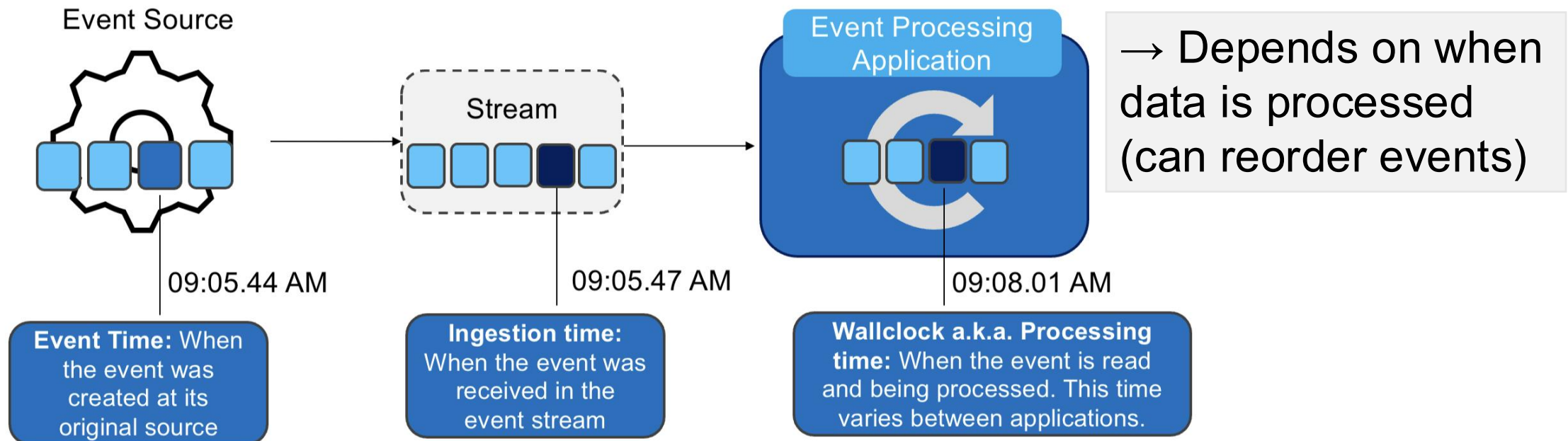
- The Event Source must include a timestamp in each Event that denotes the time when the event was created.
- On the consuming side, the Event Processing Application needs to extract the timestamp



More details on [Event-time Processing](#); Related pattern: Wallclock-time Processing

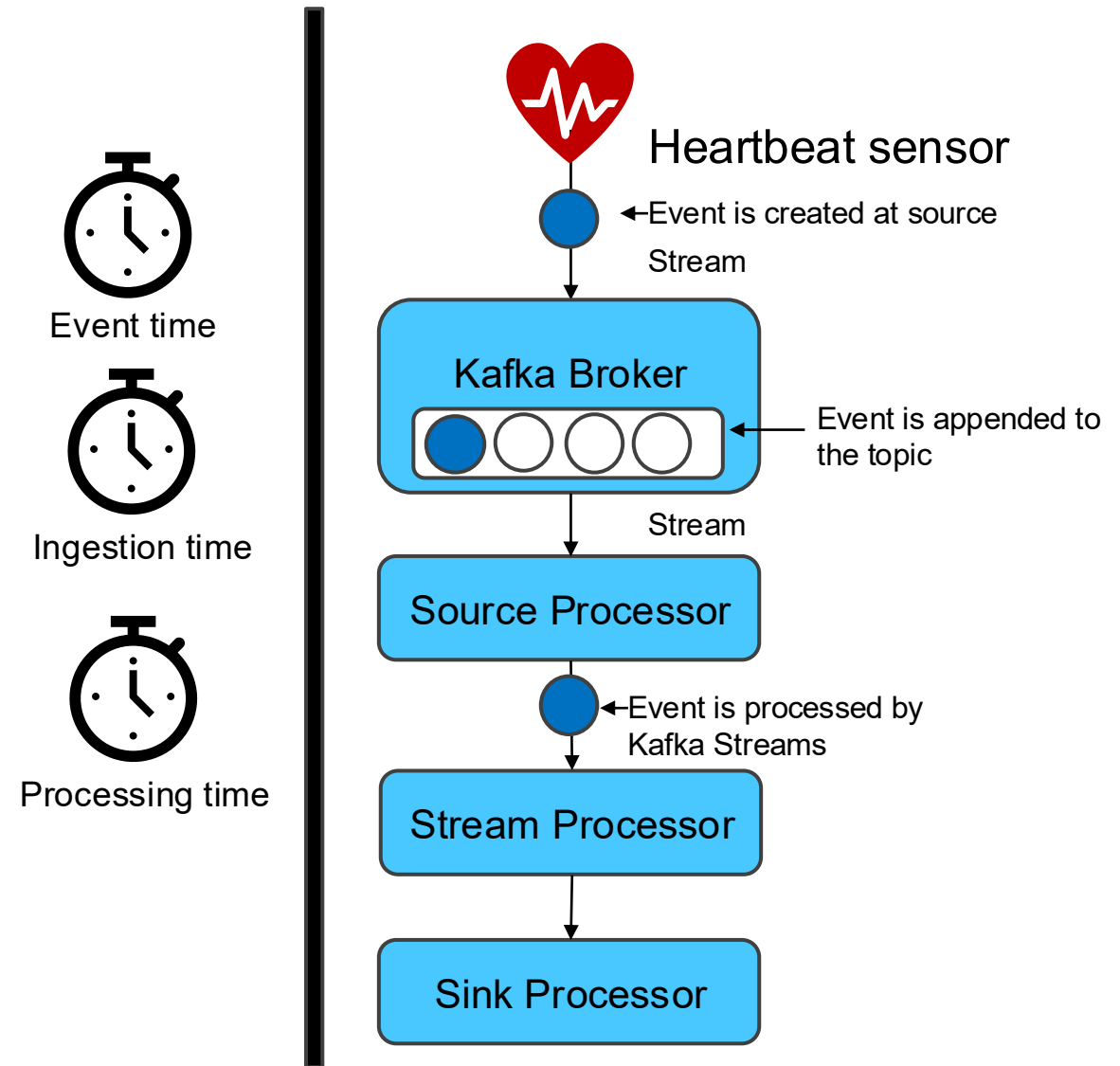
Wall-clock Time Processing

- How can events be processed irrespective of the timestamps when they were originally created by the event source?
- The event processing application can use (1) the time when the event was created, (2) the time when it was received on the event stream, (3) or a time derived from one or more fields by the event itself.



More details on [Wallclock-time Processing](#); Related pattern: Event-time processing

Different Time Semantics in Kafka Streams



Different Time Semantics in Kafka Streams

Timestamp embedded in payload

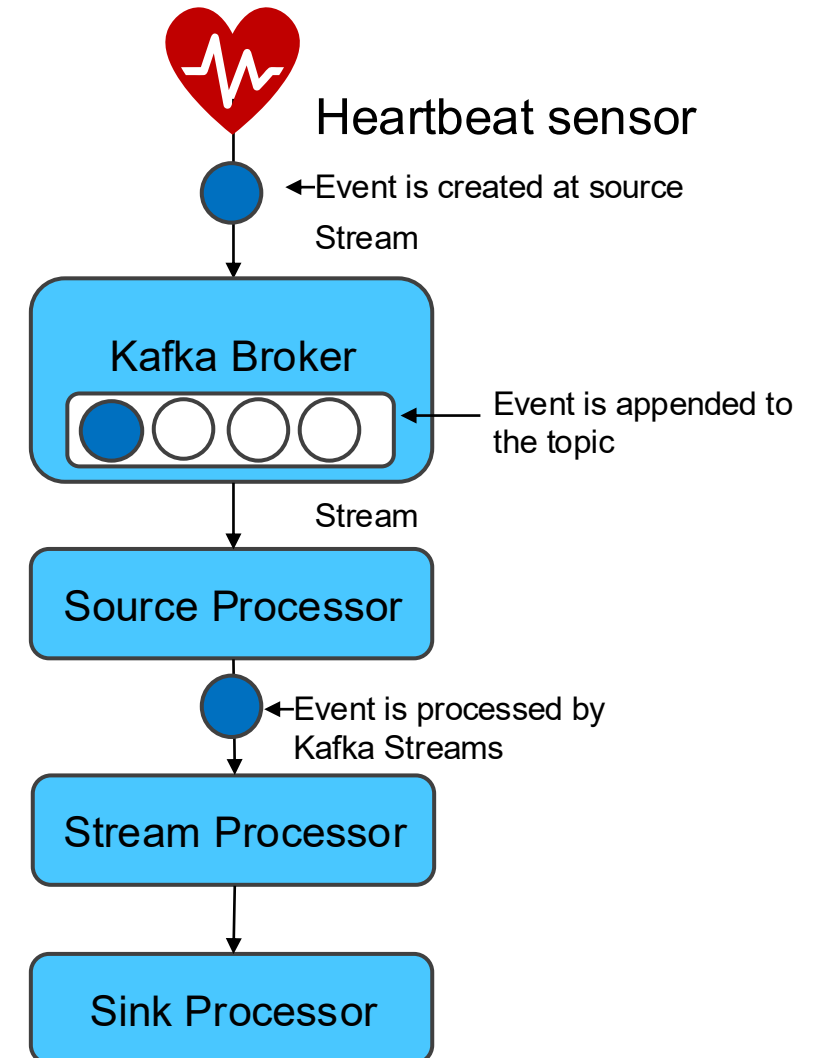
```
{  
  "timestamp": "2020-11-12T09:02:00.000Z",  
  "sensor": "smart-pulse"  
}
```

Alternatively, add default timestamps using Kafka Producer

```
log.message.timestamp.type (broker level)
```

```
message.timestamp.type (topic level)
```

There are two possible configurations for these configs: `CreateTime` and `LogAppendTime`. Note that topic level configs take precedence over broker level configs. Be sure to take `CreateTime` if you want event-time semantics.



Timestamp Extractors are responsible for associating a given record with a timestamp; your control time semantics via timestamp extraction

```
public interface TimestampExtractor {  
    long extract(  
        ConsumerRecord<Object, Object> record,  
        long partitionTime  
    );  
}
```

Kafka Streams keeps track of the most recent timestamp it has seen for each partition it consumes from, and passes it to the extract method using the partitionTime parameter.
partitionTime can be used as fallback when no timestamp can be extracted.

Included Time Extractors

- `FailOnInvalidTimestamp`: extracts timestamp from consumer record, which is either event or ingestion time
- `LogAndSkipOnInvalidTimestamp`: can also be used to achieve event time
- `WallclockTimestampExtractor`: can be used for processing time

Custom Timestamp Extractor

Custom Timestamp Extractors are common whenever event-time semantics is needed and the timestamp is included in the payload

```
public class VitalTimestampExtractor implements TimestampExtractor {  
  
    @Override  
    public long extract(ConsumerRecord<Object, Object> record, long partitionTime) {  
        Vital measurement = (Vital) record.value();  
        if (measurement != null && measurement.getTimestamp() != null) {  
            String timestamp = measurement.getTimestamp();  
            return Instant.parse(timestamp).toEpochMilli();  
        }  
        return partitionTime;  
    }  
}
```

Strategies when no timestamp can be extracted:

- Throw an exception and stop processing
- Fallback to partitionTime
- Return a negative timestamp

Registering Streams With a Timestamp Extractor

```
StreamsBuilder builder = new StreamsBuilder();

Consumed<String, Pulse> pulseConsumerOptions =
    Consumed.with(Serdes.String(), JsonSerializer.Pulse())
    .withTimestampExtractor(new VitalTimestampExtractor());

KStream<String, Pulse> pulseEvents =
    builder.stream("pulse-events", pulseConsumerOptions);

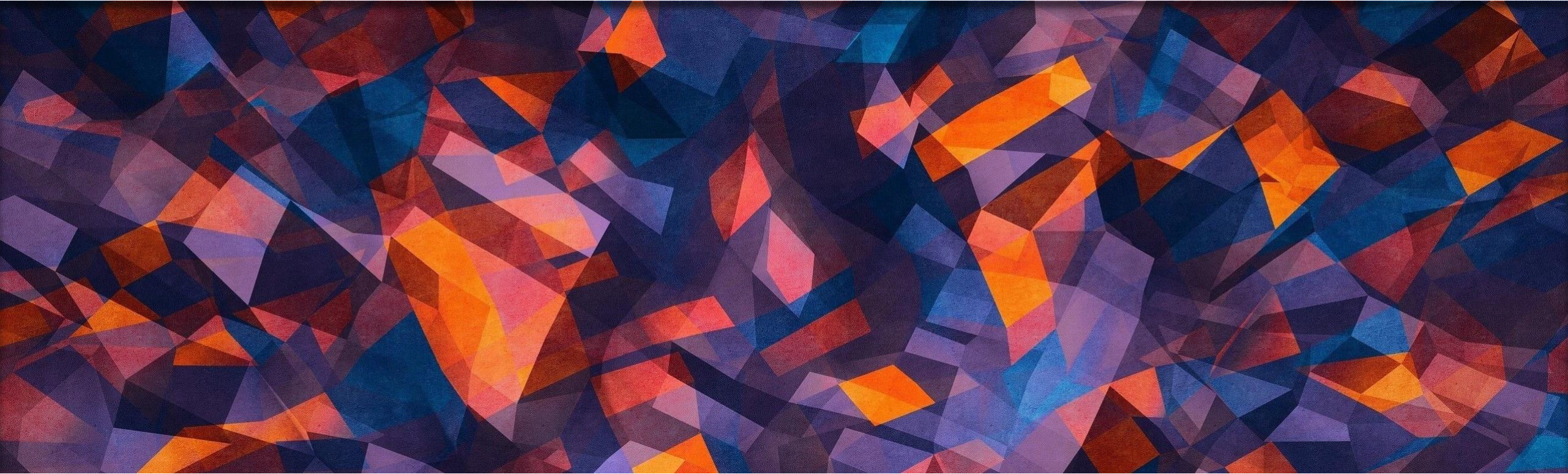
Consumed<String, BodyTemp> bodyTempConsumerOptions =
    Consumed.with(Serdes.String(), JsonSerializer.BodyTemp())
    .withTimestampExtractor(new VitalTimestampExtractor());

KStream<String, BodyTemp> tempEvents =
    builder.stream("body-temp-events", bodyTempConsumerOptions);
```

Use `Consumed.withTimestampExtractor` to tell Kafka Streams to use the custom timestamp extractor (`VitalTimestampExtractor`) for extracting vitals timestamps.



Universität St.Gallen



Event-driven and Process-oriented Architectures

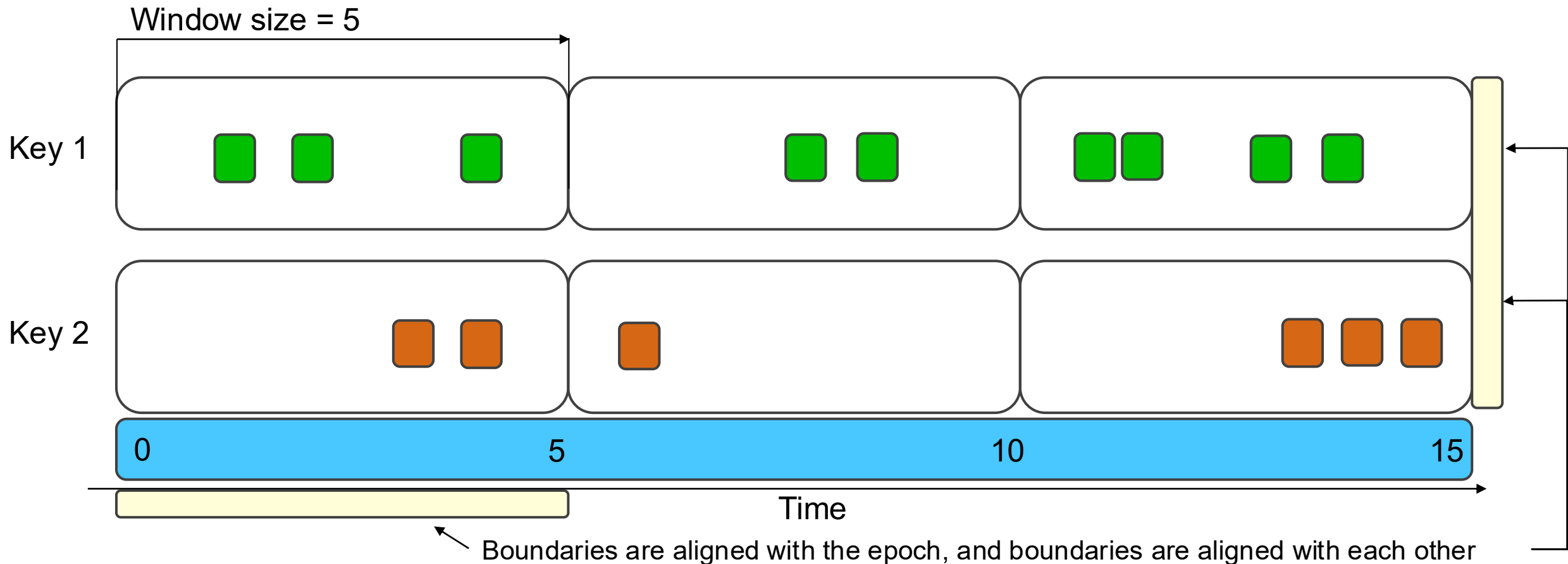
Introduction into Windows

Lecture 10

- Windows define how we group events in time
 - Event time semantics: records that occurred around the same time
 - Processing time semantics : records that were processed around the same time
- Window Types
 - Tumbling windows
 - Hopping Windows
 - Session Windows
 - Sliding join window
 - Sliding aggregation windows
- Choice of window type affects result

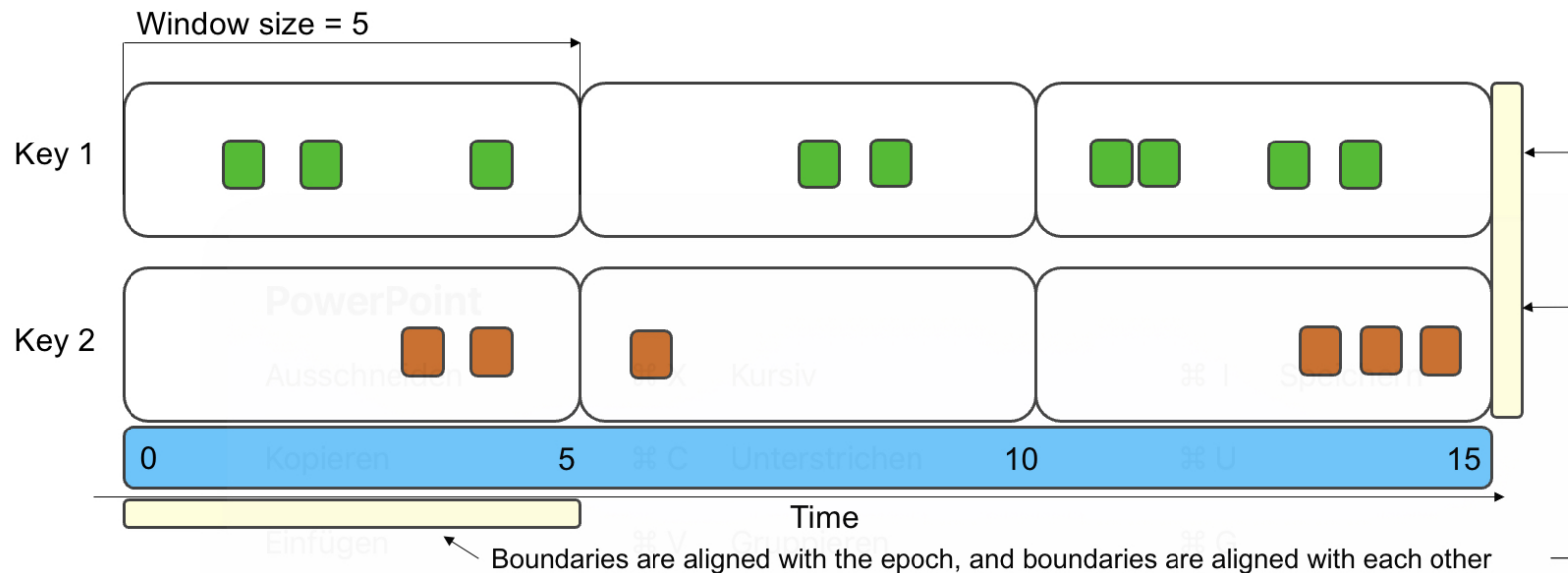
Tumbling Windows

- Fixed-size windows that never overlap
 - Single property: window size
- like fixed time buckets (e.g., every 5s)



Tumbling Windows

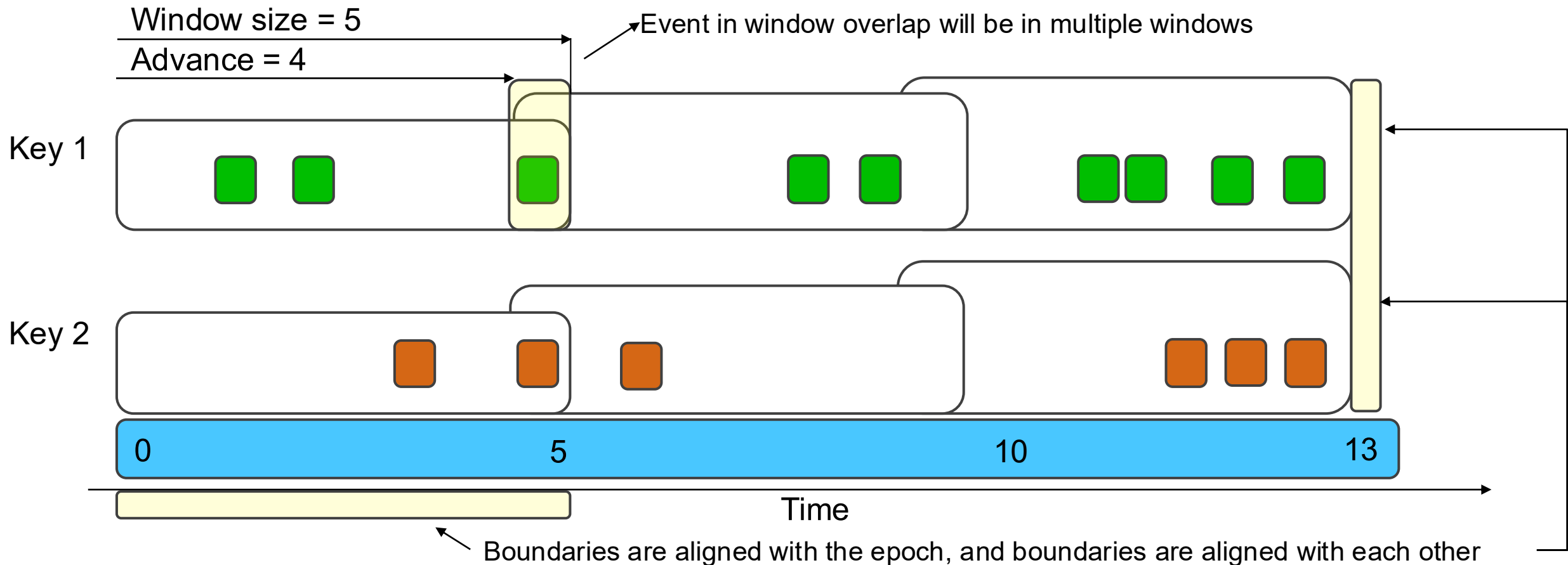
- Fixed-size windows that never overlap
 - Single property: window size
- like fixed time buckets (e.g., every 5s)



```
TimeWindows tumblingWindow =  
    TimeWindows.ofSizeWithNoGrace(Duration.ofSeconds(5));
```

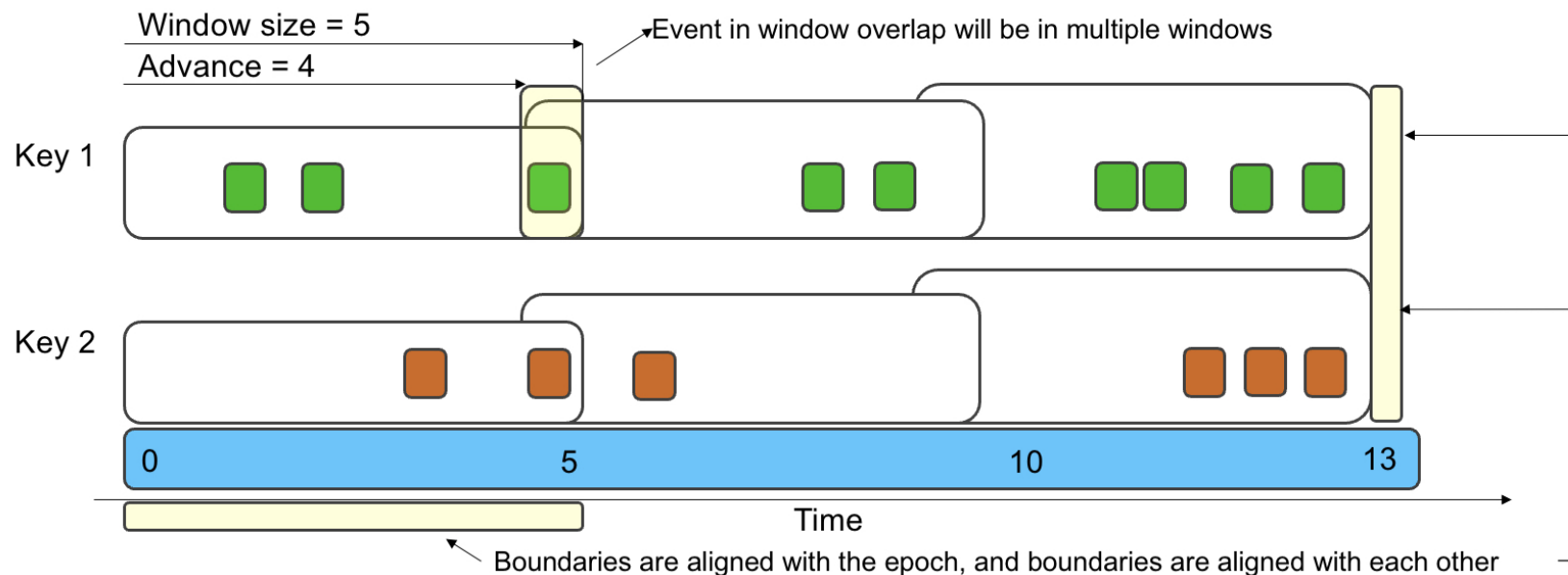
Hopping Windows

- Fixed-size windows that may overlap
 - Two properties: window size and advance interval
- Same event can belong to multiple windows



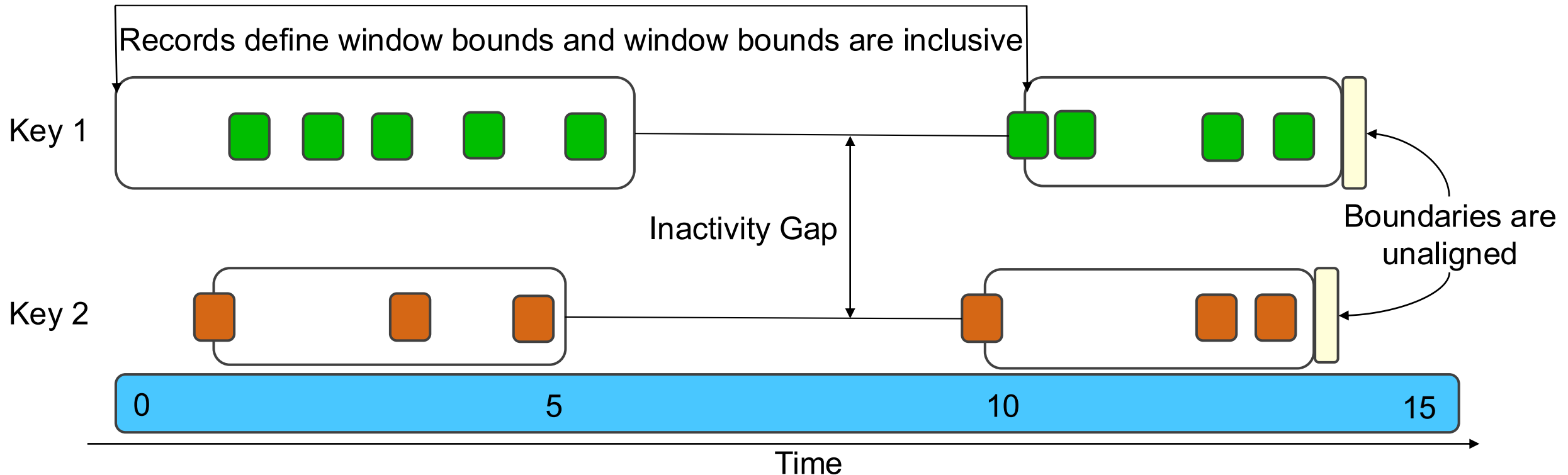
Hopping Windows

- Fixed-size windows that may overlap
 - Two properties: window size and advance interval
- Same event can belong to multiple windows

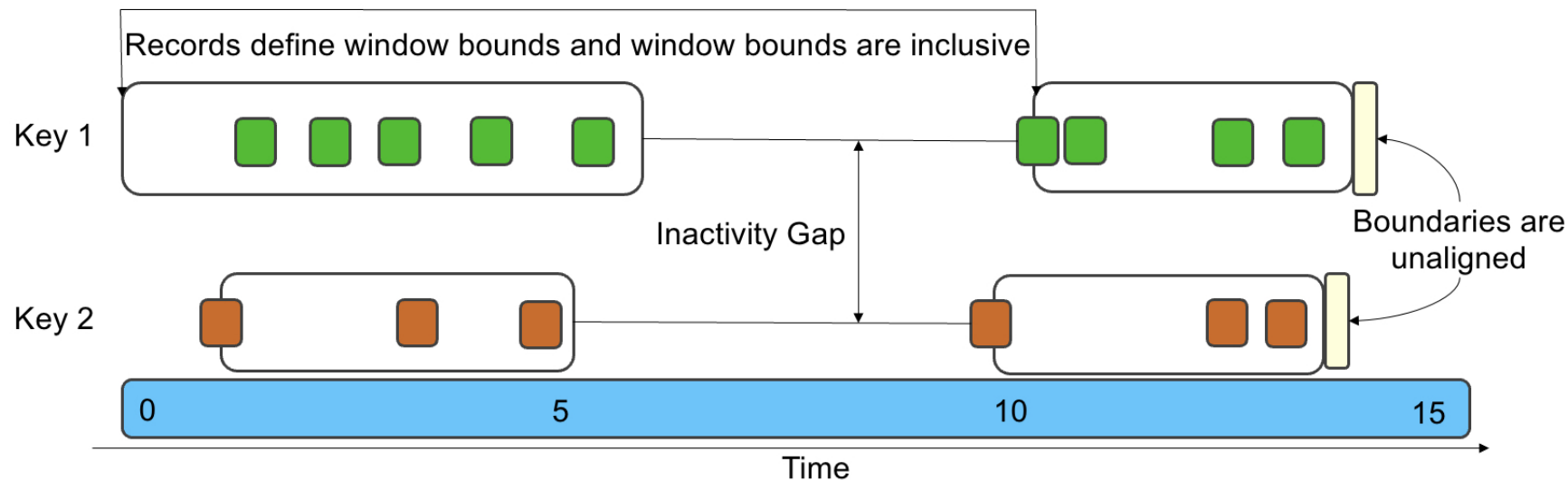


```
TimeWindows hoppingWindow = TimeWindows
    .ofSizeWithNoGrace(Duration.ofSeconds(5))
    .advanceBy(Duration.ofSeconds(4));
```

- Variable-sized windows that are determined by periods of activity followed by gaps of inactivity
 - Single parameter: inactivity gap
- Windows grow dynamically based on activity



- Variable-sized windows that are determined by periods of activity followed by gaps of inactivity
 - Single parameter: inactivity gap
- Windows grow dynamically based on activity

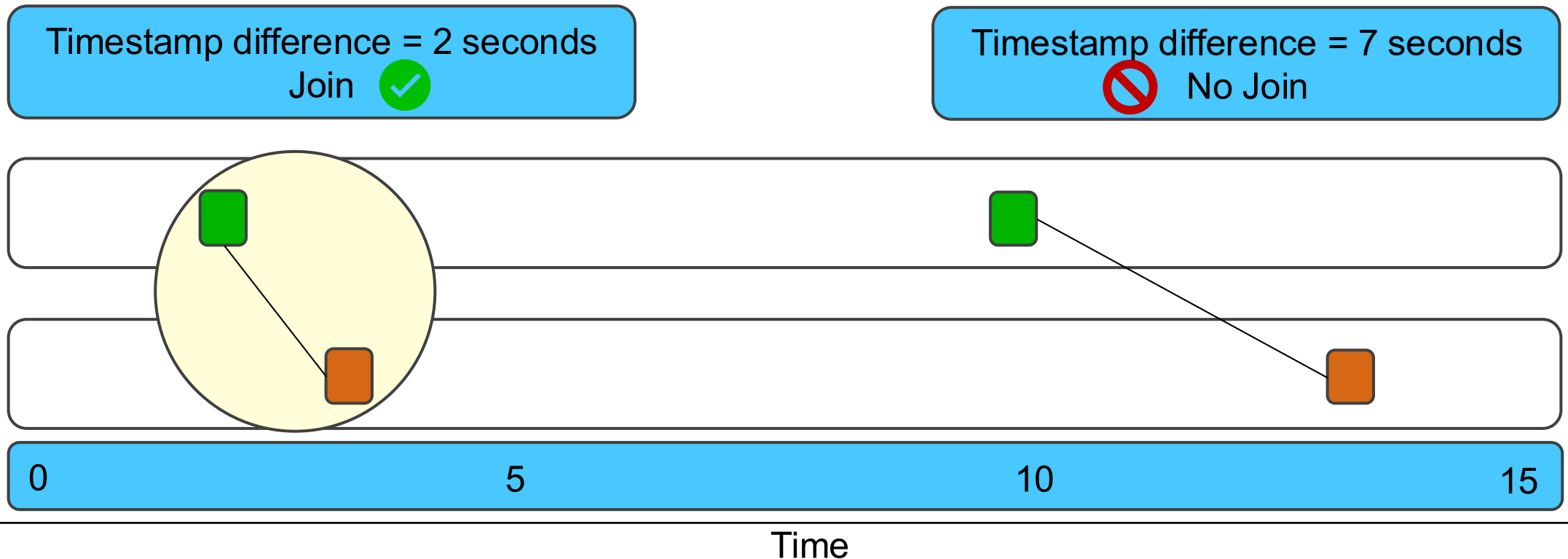


```
SessionWindows sessionWindow = SessionWindows  
    .ofInactivityGapWithNoGrace(Duration.ofSeconds(5));
```

Sliding Join Windows

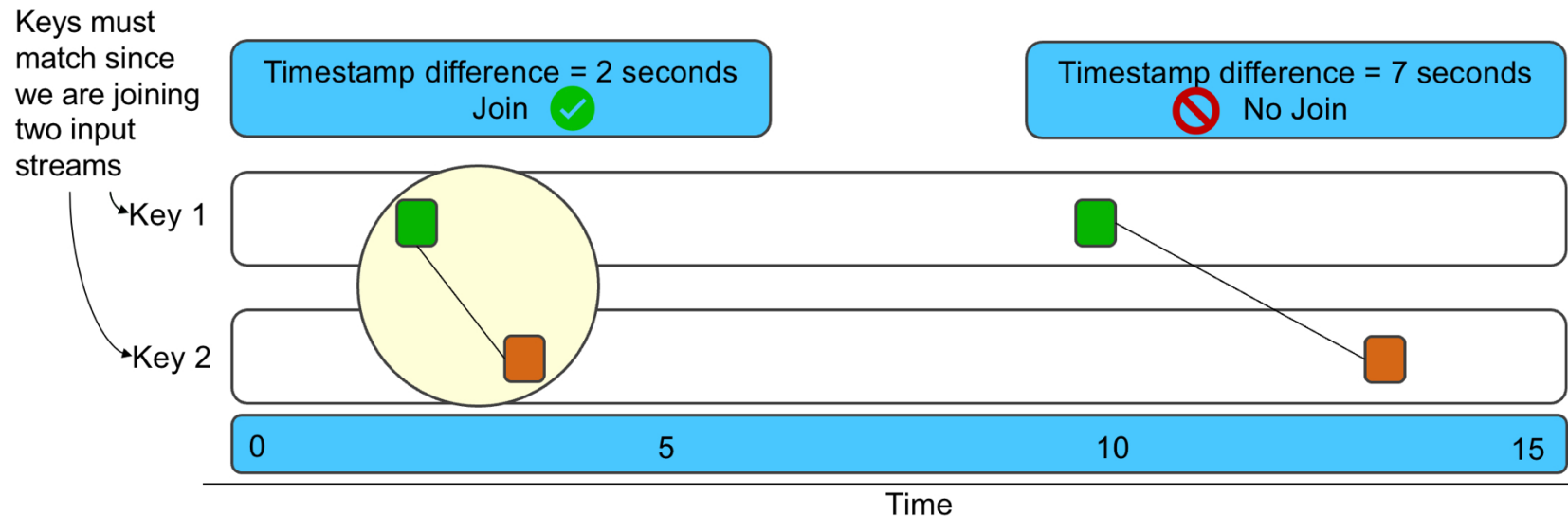
- Fixed-size used for joins and created using the JoinWindows class
- Two records are joined if their timestamp differ by $\leq$ window size $\rightarrow$ Join if events happen close in time

Keys must match since we are joining two input streams



Sliding Join Windows

- Fixed-size used for joins and created using the JoinWindows class
- Two records are joined if their timestamp differ by $\leq$ window size $\rightarrow$ Join if events happen close in time



```
JoinWindows joinWindow = JoinWindows  
    .ofTimeDifferenceWithNoGrace(Duration.ofSeconds(5)) ;
```

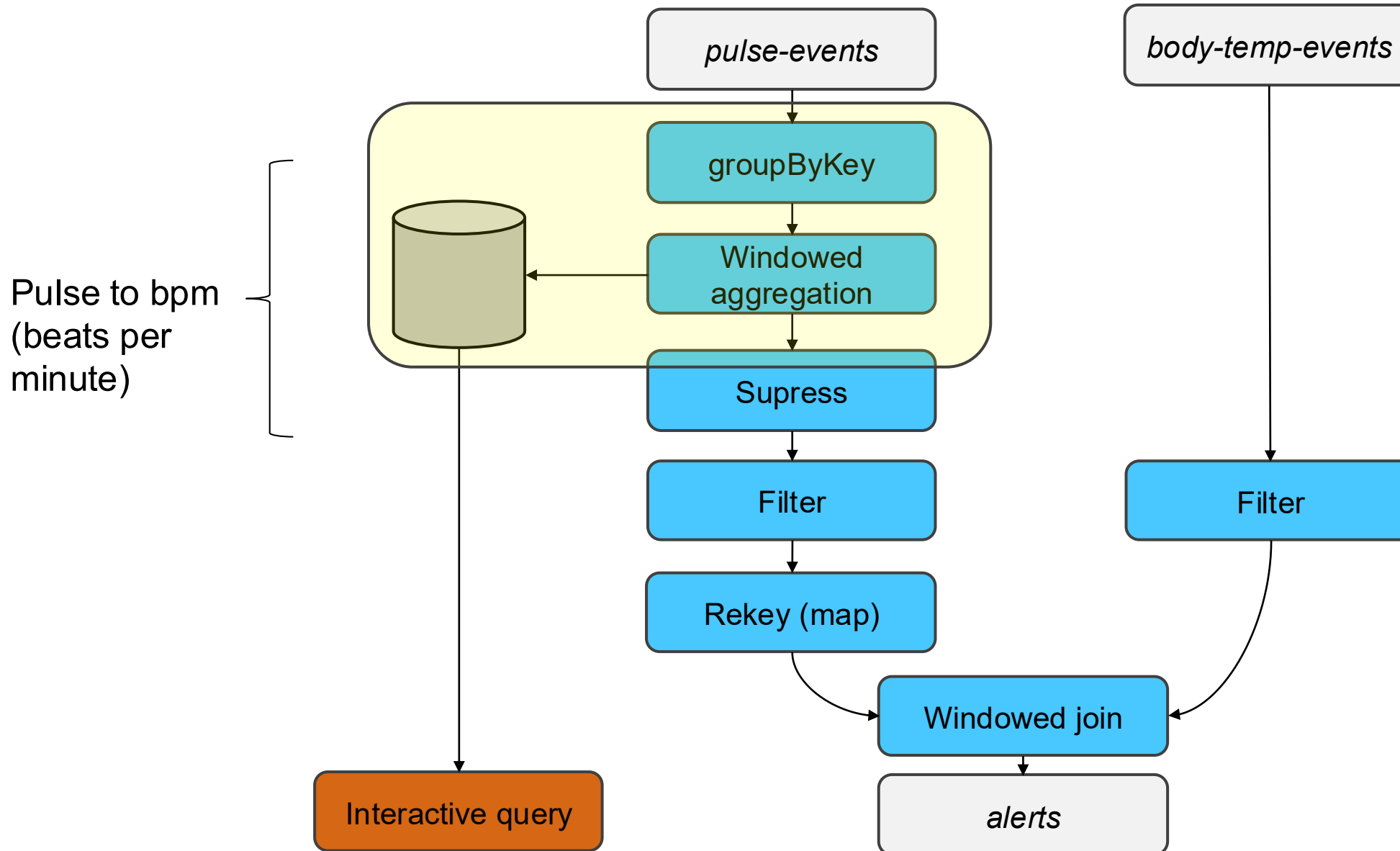
Sliding Aggregation Windows

- No fixed buckets (window defined per record)
 - Events are grouped if their timestamps are close (within window size)
- Continuous / fine-grained aggregation

```
SlidingWindows slidingWindow =  
    SlidingWindows.ofTimeDifferenceAndGrace(  
        Duration.ofSeconds(5),  
        Duration.ofSeconds(0));
```


Example Topology

Time-aware Stream Processing



Selecting a Window

- Which window type is appropriate to convert raw pulse events into heart rate using a windowed aggregation?
 - **Tumbling windows:** → fixed 60s buckets
 - **Hopping windows:** → overlap not needed
 - **Session windows:** → variable size, not appropriate
 - **Sliding join window:** → for joins only
 - **Sliding aggregation windows:** → no fixed buckets



```
TimeWindows tumblingWindow =  
    TimeWindows.ofSizeWithNoGrace(Duration.ofSeconds(60));
```

Window Aggregations

```
TimeWindows tumblingWindow =  
    TimeWindows.ofSizeWithNoGrace (Duration.ofSeconds (60)) ;  
  
KTable<Windowed<String>, Long> pulseCounts =  
    pulseEvents  
        .groupByKey ()  
        .windowedBy (tumblingWindow)  
        .count (Materialized.as ("pulse-counts")) ;  
  
pulseCounts  
    .toStream ()  
    .print (Printed.<Windowed<String>, Long>toSysOut ().withLabel ("pulse-counts")) ;
```

Key of KTable changed from
String to Windowed<String>

- **Note:** Results are written to output stream whenever a new event arrives
→ result is continuously updated per window

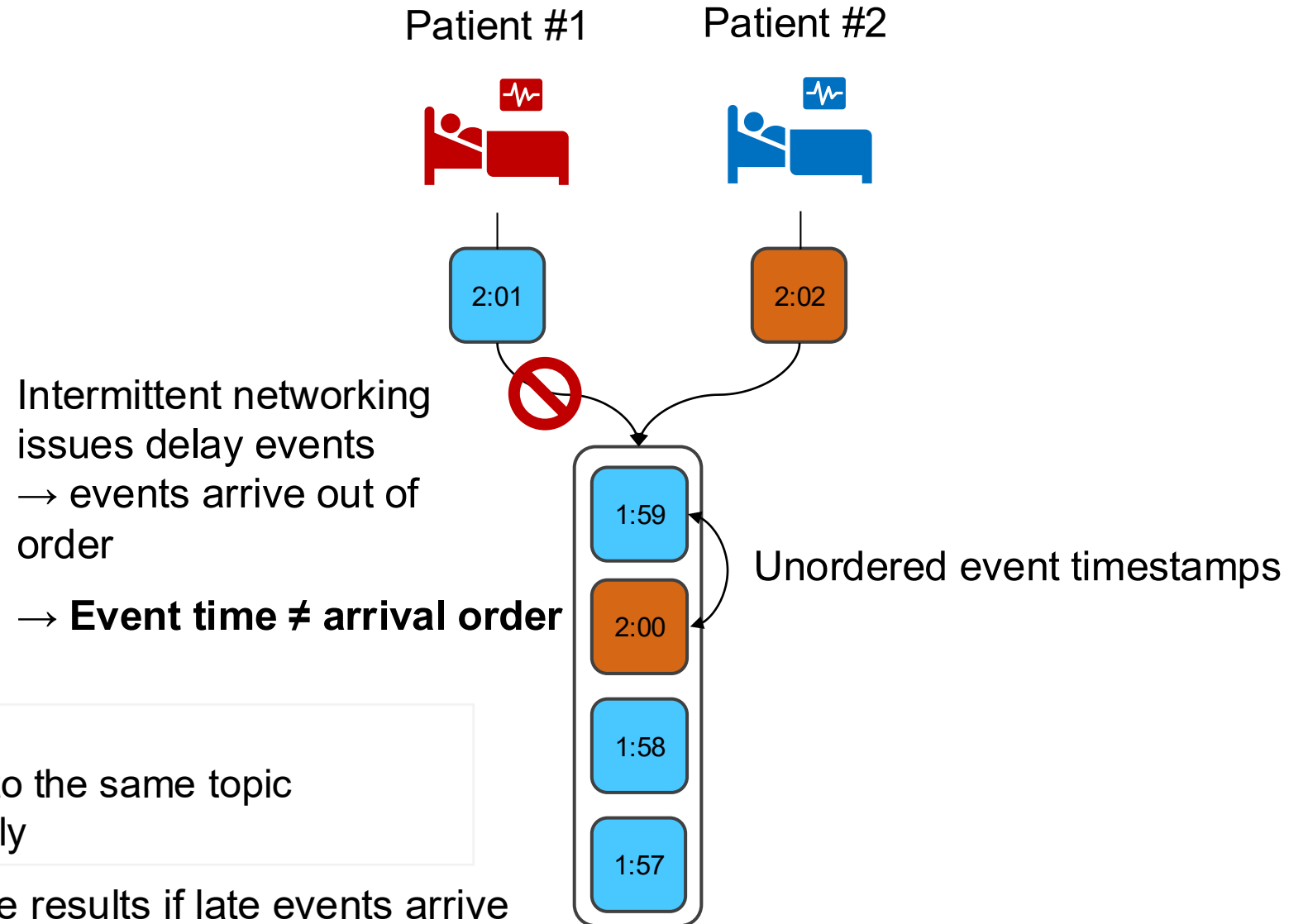
Emitting Window Results

- When should window results be emitted?
 - More complex than it seems
- Unbounded event streams may **not** be in *timestamp order*
 - Event-time processing $\neq$ arrival order
- Kafka guarantees order per partition (*offset order*) → NOT by event-time
- Events can arrive late (**out-of-order events**) → earlier events may arrive after later ones
- Tradeoff:
 - Wait longer → more complete results
 - Emit earlier → lower latency

Key takeaway:

→ Results may change over time as late events arrive

The Problem of Out-of-Order Events



Example:

Multiple producers send events to the same topic

→ events interleave unpredictably

→ Windows may miss or update results if late events arrive

- One of the biggest challenges: handling **delayed events**
- Kafka Streams: define how long to wait for late events (**grace period**)
→ Wait for late events before closing the window
- Keeps the window open to include delayed / out-of-order events
- Other systems (e.g., Flink) use watermarks for the same purpose
- Tradeoff:
 - longer grace → more complete results
 - Shorter grace → lower latency

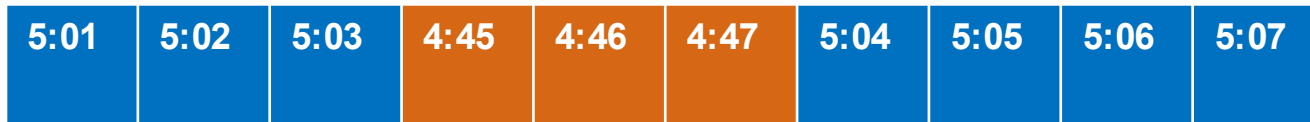
```
TimeWindows tumblingWindow =  
    TimeWindows.ofSizeAndGrace(Duration.ofSeconds(60),  
        Duration.ofSeconds(5));
```

Event Processing Design Patterns

Out-of-Sequence Events

Out-of-sequence are common in IoT scenarios

Example: Device loses connection; sends buffered events later → older events arrive late



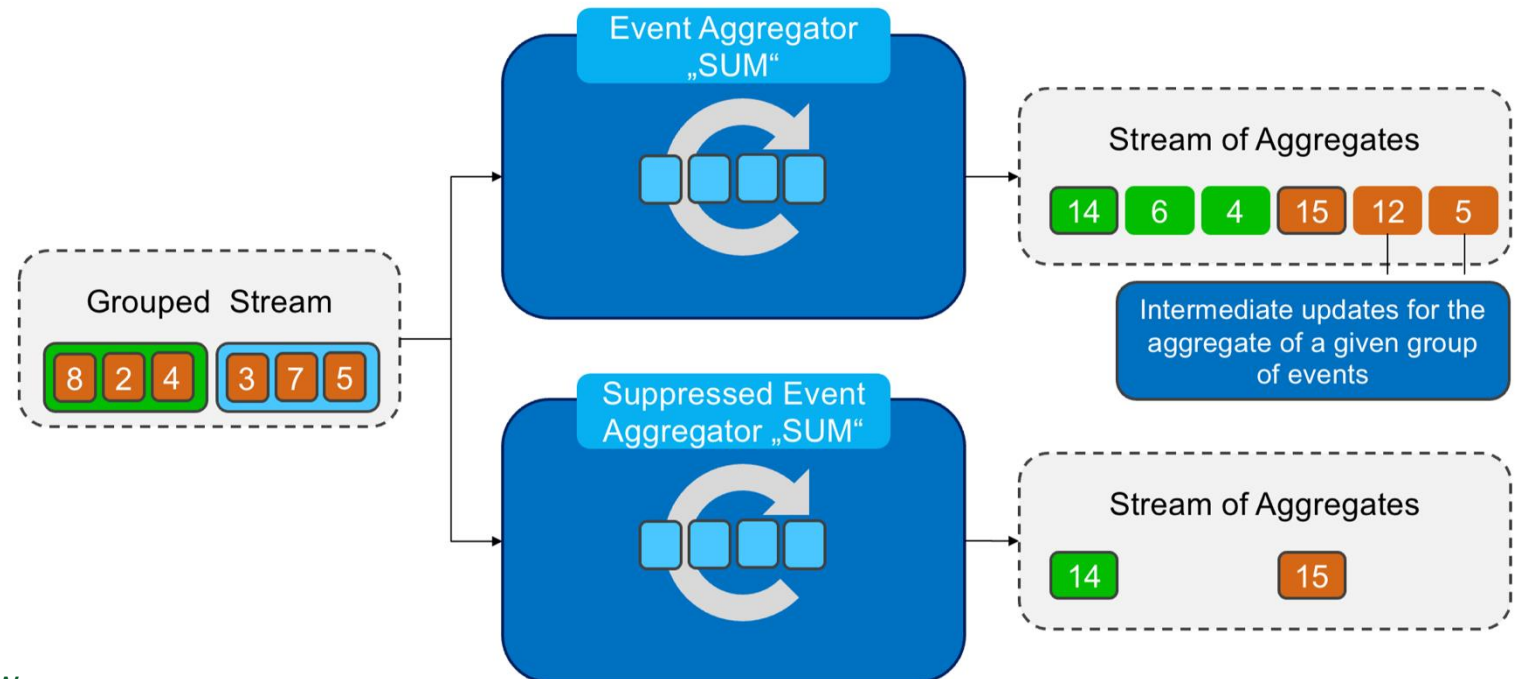
Older events arriving late

This requires applications to 1) recognize that an event is out of order by examining the event time, 2) attempt to reconcile out-of-sequence events during a grace period

Suppressed Event Aggregator

Supressed Event Aggregator. Provides final aggregation results, while omitting intermediate results

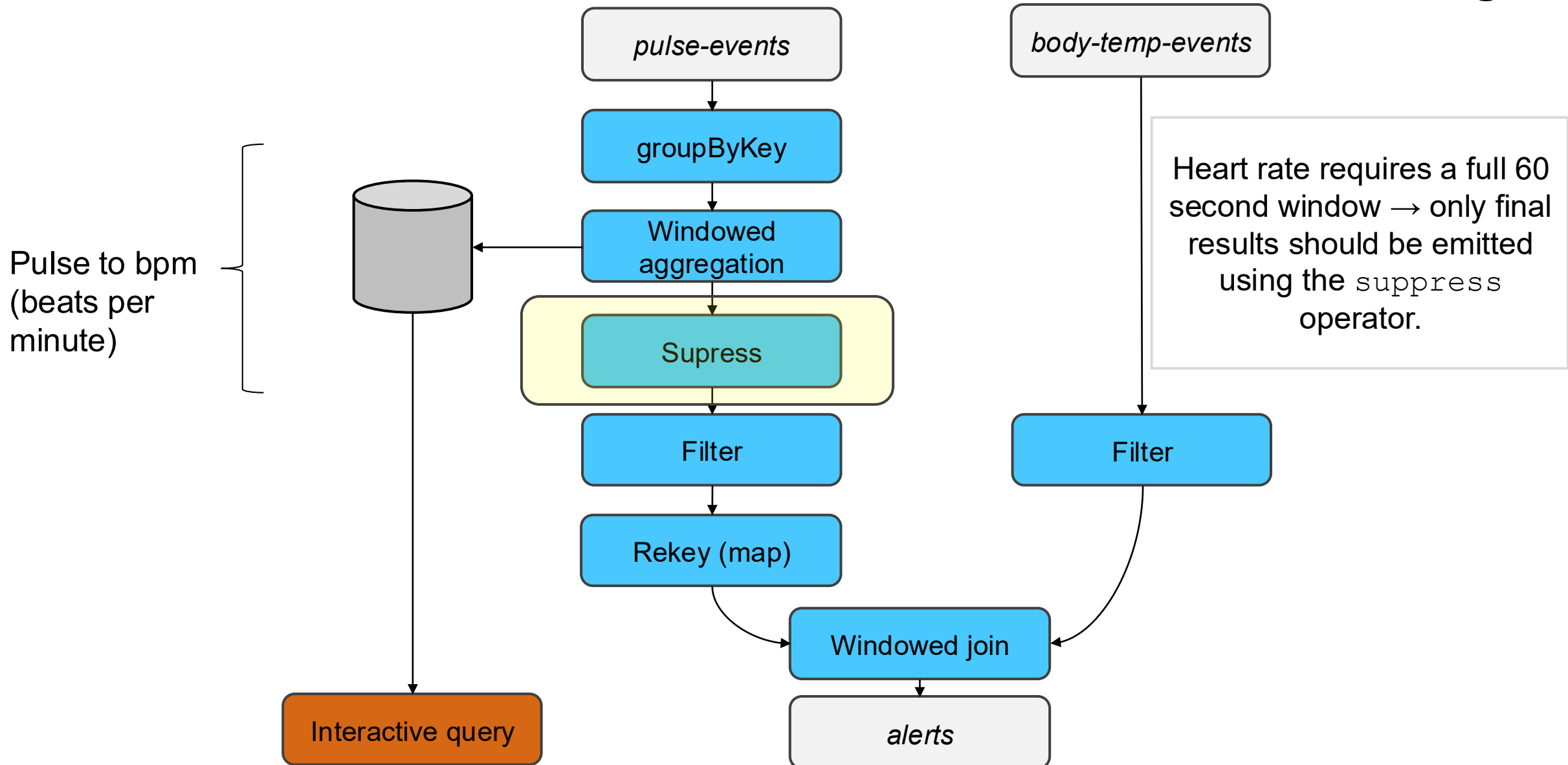
- Requires a grouped stream as input where events are grouped into windows based on timestamps
- Emits a single final event when the window closes (no intermediate results)
- Ideally, is able to handle delayed events



Tutorial: [Emit final results from a time window](#)

Example Topology

Time-aware Stream Processing



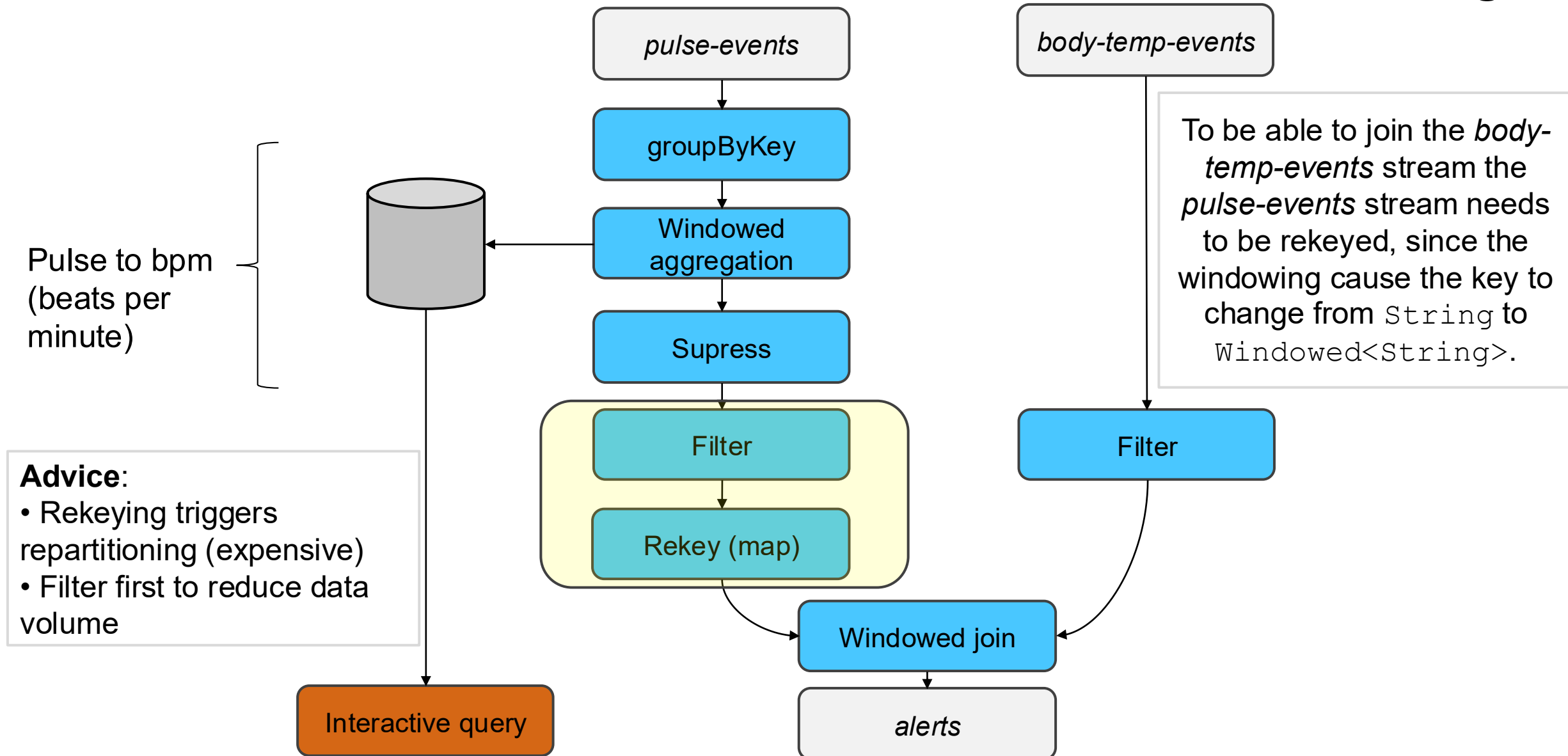
- Buffers intermediate results
- Emits only final results (e.g., when window closes)
- Requires:
 - Buffering (memory)
 - Strategy for handling overflow

- Kafka Streams allows to suppress intermediate events using the `suppress` operator

```
TimeWindows tumblingWindow =  
    TimeWindows  
        .ofSizeAndGrace(Duration.ofSeconds(60),  
            (Duration.ofSeconds(5)));  
  
KTable<Windowed<String>, Long> pulseCounts =  
    pulseEvents  
        .groupByKey()  
        .windowedBy(tumblingWindow)  
        .count(Materialized.as("pulse-counts"))  
        .suppress(  
            Suppressed.untilWindowCloses(BufferConfig.unbounded().shutdownWhenFull()));
```

Example Topology

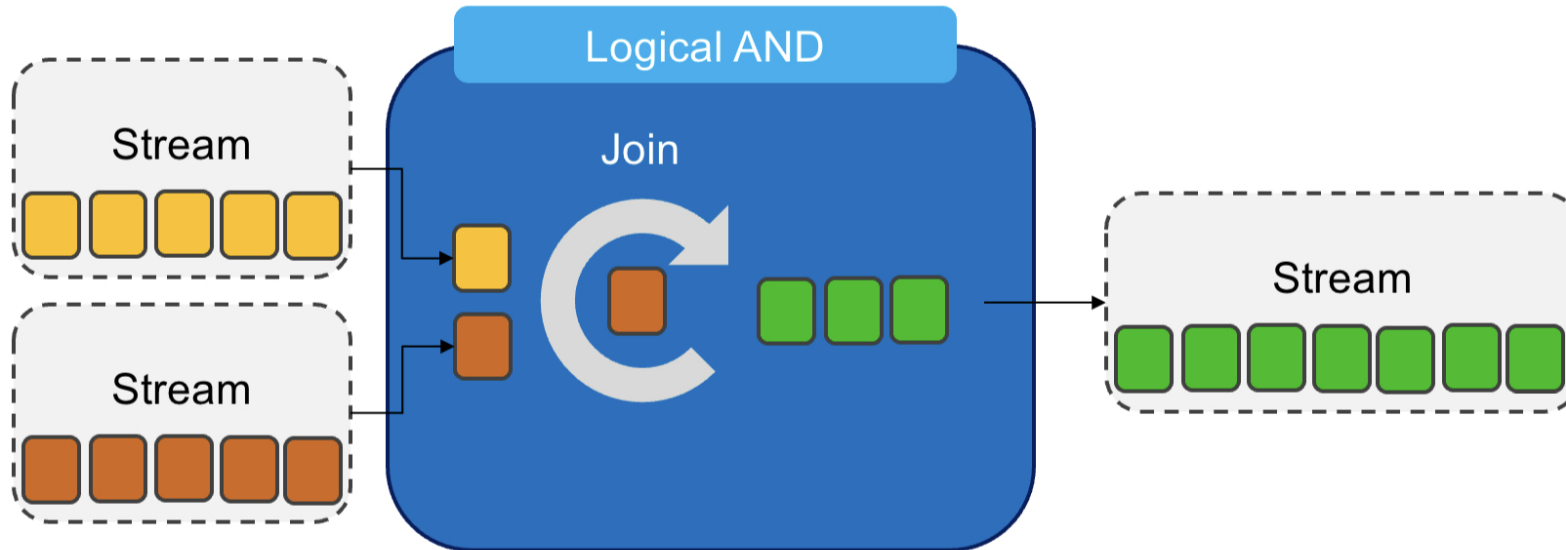
Time-aware Stream Processing



Logical AND = Windowed Join Across Stream

Logical AND. Allows to combine information from several event streams

- Has several input streams and one output stream
- Streams are joined based on a common key
- Require a time window and events on both sides of the join need to arrive within the window

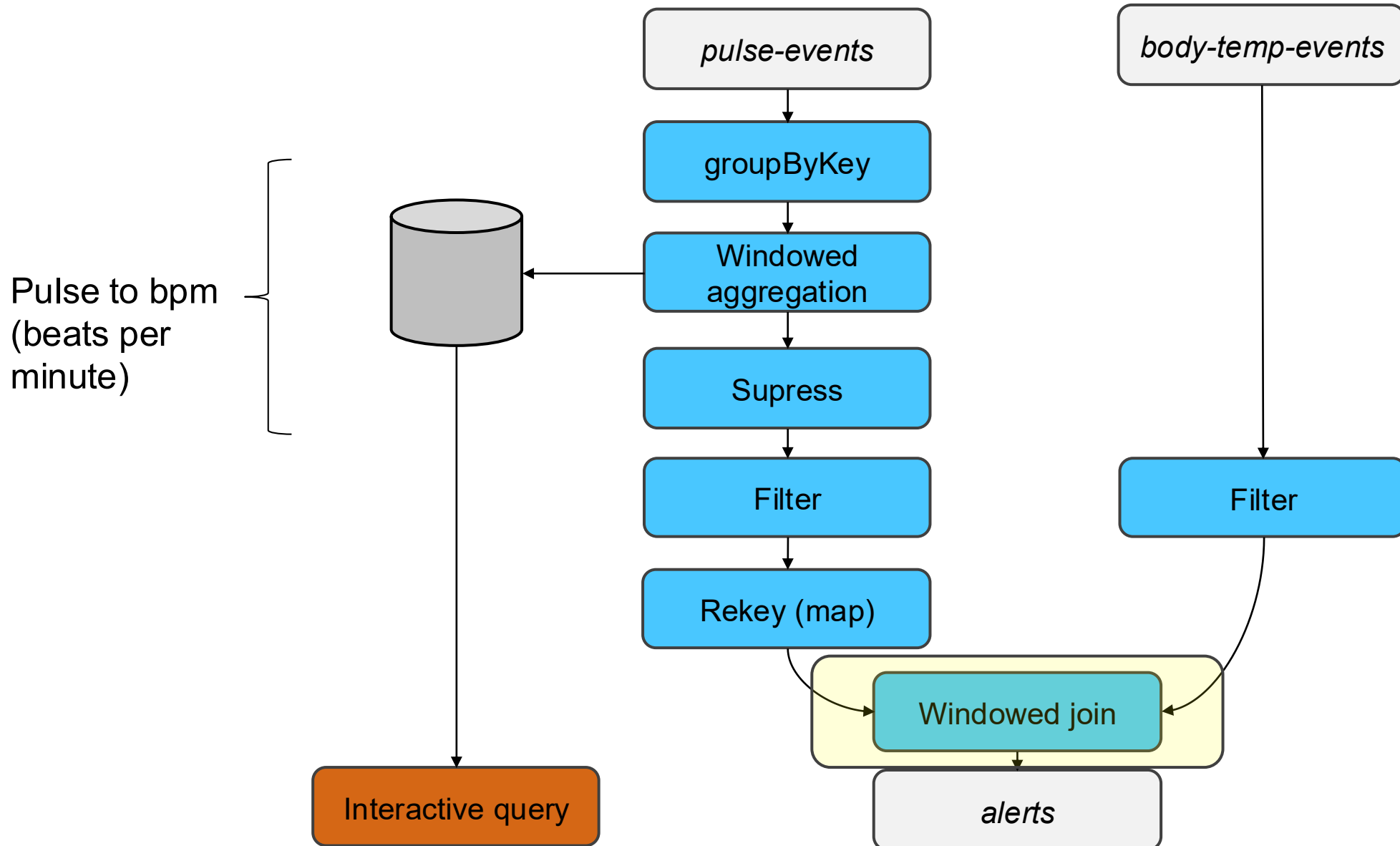


See also the [Projection Table](#), a memory-efficient way of considering an Event Stream over a potentially-infinite time period

- Windowed joins require a **sliding window**
- Sliding window joins compare the timestamps at both sides of the join to determine which records should be joined together
- `KStream-KStream` joins must be **windowed** since streams are **unbounded**
- Data needs to be **materialized into a local state** store for performing quick lookups of related values
- Each stream produces events at different rates
→ requires **synchronization** via time windows

Example Topology

Time-aware Stream Processing



Windowed Joins

```
StreamJoined<String, Long, BodyTemp> joinParams =  
    StreamJoined.with(Serdes.String(), Serdes.Long(), JsonSerializer.BodyTemp());
```

Defines the Serdes to be used by the join

```
JoinWindows joinWindows =  
    JoinWindows.ofTimeDifferenceAndGrace  
        (Duration.ofSeconds(60),  
         Duration.ofSeconds(10));
```

Records with 60 seconds apart or less will fall into the same window and be joined

Tolerate a delay up to 10 seconds

```
ValueJoiner<Long, BodyTemp, CombinedVitals> valueJoiner =  
    (pulseRate, bodyTemp) -> new CombinedVitals(pulseRate.intValue(), bodyTemp);
```

Heart rate and body temperature are combined in a CombinedVital object

```
KStream<String, CombinedVitals> vitalsJoined =  
    highPulse.join(highTemp, valueJoiner, joinWindows, joinParams);
```

Perform join

Event Processing Design Patterns

Streaming Join

When we join two streams, we conceptually join the entire history, try to match events in one stream with events in the other stream that have the same key and happened in the same time window (*windowed join*).

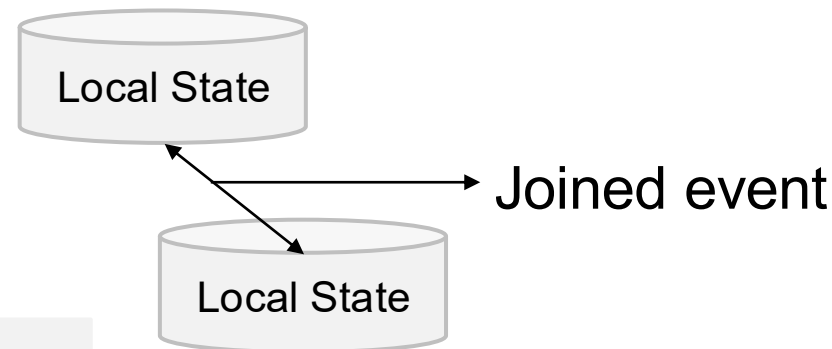
Clicks



Five-second window

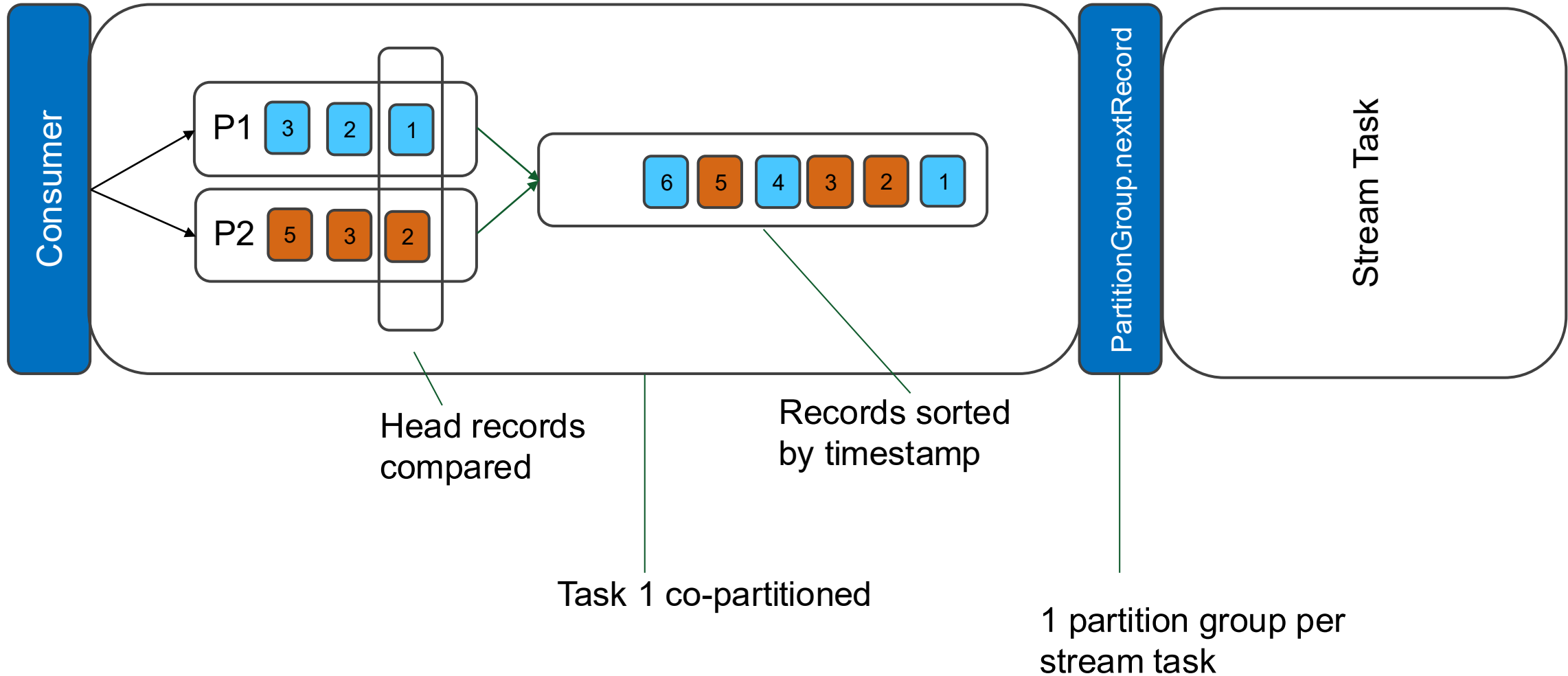
Five-second window

Searches



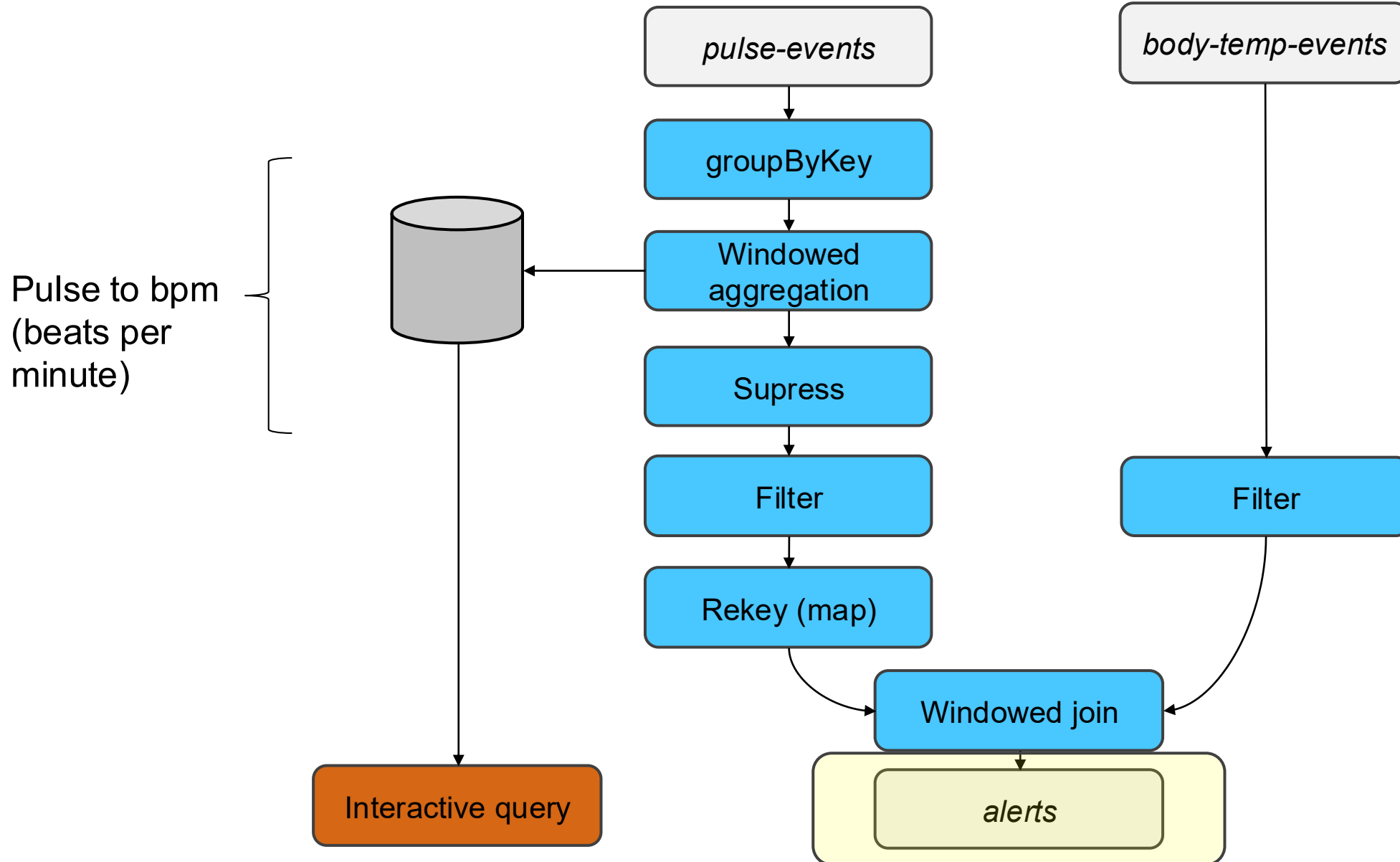
- Time controls processing order (not arrival order)
- Kafka Streams synchronizes input streams:
 - processes the record with the lowest timestamp first
- This ensures correct results for joins and aggregations

Time-driven Data Flow



→ processing order = timestamp order

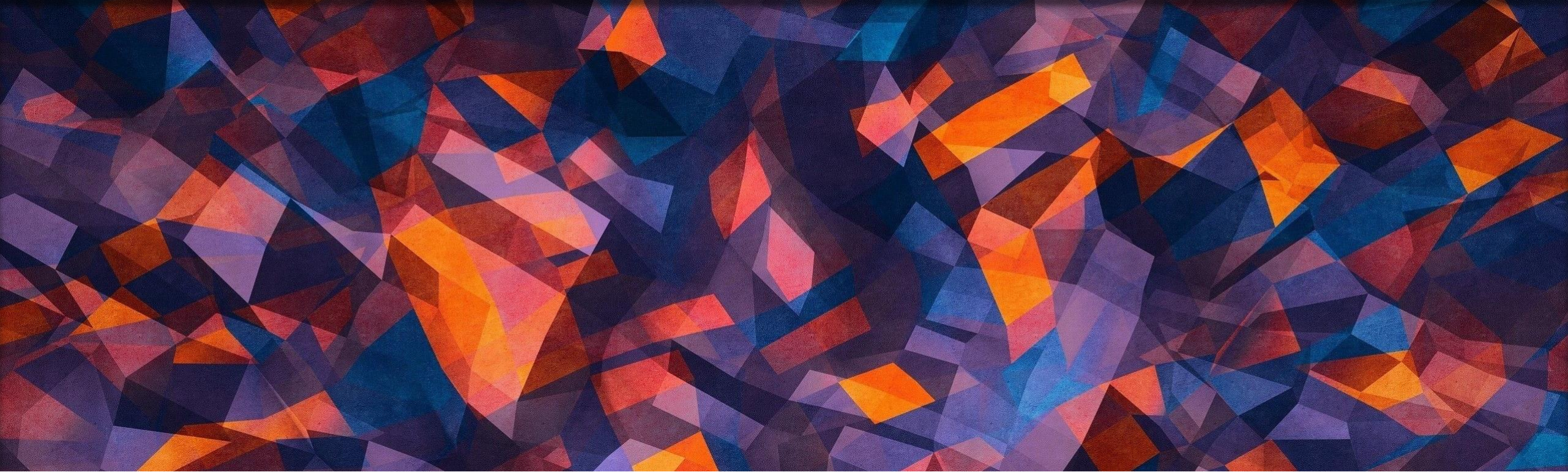
Example Topology Alerts Sink



- Join results are written to an output topic (*alerts*)
- Output records keep a timestamp
→ usually derived from input events
- For joins:
→ timestamp = max of input timestamps

Querying Windowed Stores

- Keys include:
 - original key + time window
 - Queries:
 - Lookup by key within a time range
 - Scan keys within a time range
 - Iterate over all entries
- Time range is required for key-based queries



Event-driven and Process-oriented Architectures

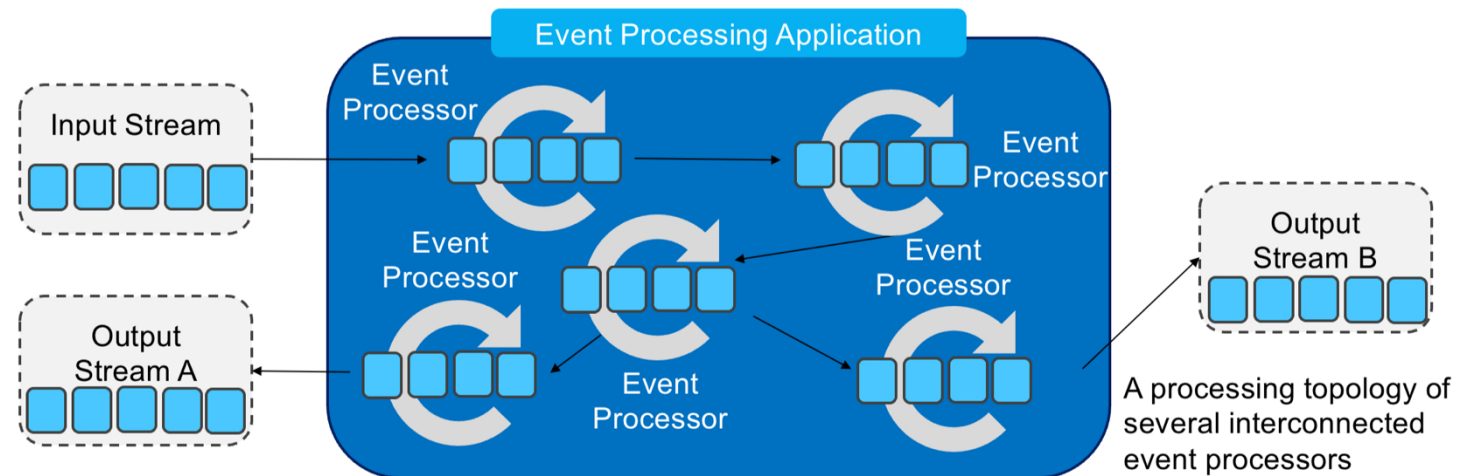
Kafka Streams Processing API and ksqlDB

Lecture 10

Kafka Streams DSL versus Processor API

- The processor topology of an Event Processing Application can be written using the Kafka Streams API
 - Kafka Streams DSL: A high-level API that provides the most common event processors out of the box (focus in the course)
 - Processor API: A low-level API that allows you to add and connect processors as well as to interact directly with state stores

DSL = 90% of use cases, Processor API = when you need full control



Example: [Processor for comparing two consecutive events in a Kafka Stream](#)

Use Processor API when DSL is not expressive enough

- **Advantages**

- Access to record metadata (topic, partition, offset, headers, etc.)
- Ability to schedule periodic functions
- Fine-grained control over when records are forwarded
- Direct and flexible access to state stores
- → Work around DSL limitations (custom logic)

- **Disadvantages**

- More verbose code → harder to read and maintain
- Higher barrier to entry for developers
- More error-prone (reinventing DSL functionality, performance pitfalls)

The Event Streaming Database ksqlDB

- ksqlDB is an event streaming database released in 2017 by Confluent
- Integrates Kafka Streams and Kafka Connect and provides a high-level SQL interface for interacting with these components
 - Model data as either **streams** or **tables** using SQL
 - Use **SQL for processing** streams and tables (e.g., joining, aggregating, filtering, transforming, and windowing) to create new, derived representations
 - Use **push queries**, which run continuously and emit new results when new data arrives (works well together with event-driven microservices)
 - Create **materialized views** from streams and tables and query them using **pull queries**
 - **Define connectors** to integrate ksqlDB with external data sources

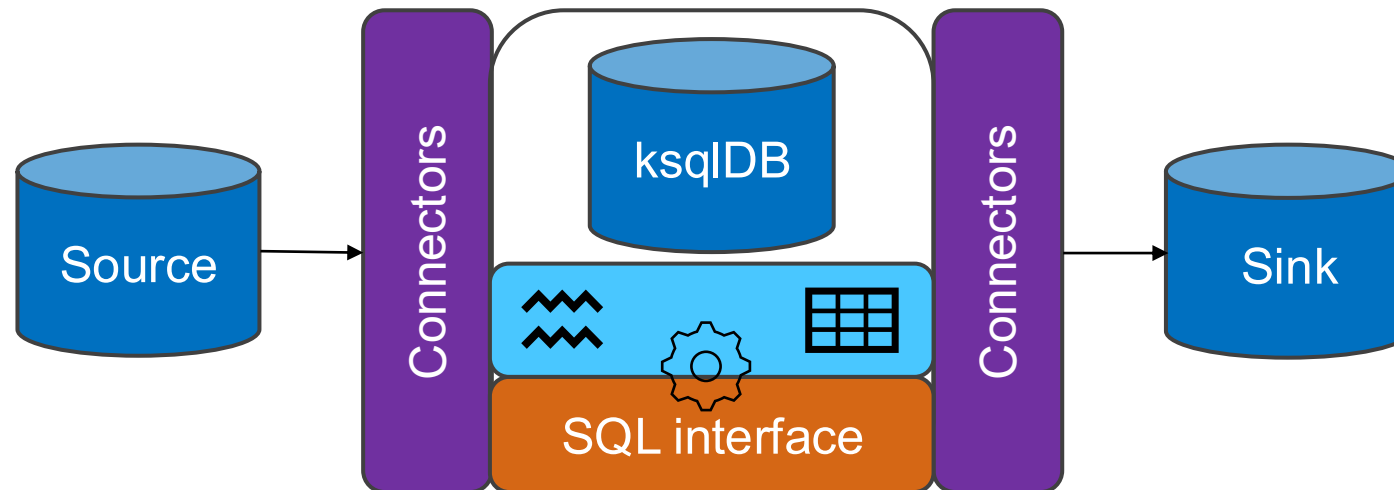
When to use ksqlDB?

- More interactive workflows through an included CLI and REST service for submitting queries
- Less code to maintain since stream processing topologies are expressed in SQL rather than a JVM language
- Lower barriers to entry, especially when already familiar with SQL
- Simplified architecture: interface for managing connectors and transforming data combined
- Increased developer productivity: less code to express topologies and complexity of lower-level systems hidden
- Cross-project consistency
- Easy setup and turnkey deployments
- Better support for data exploration

Rule of thumb:→ Use ksqlDB when problems can be expressed naturally in SQL

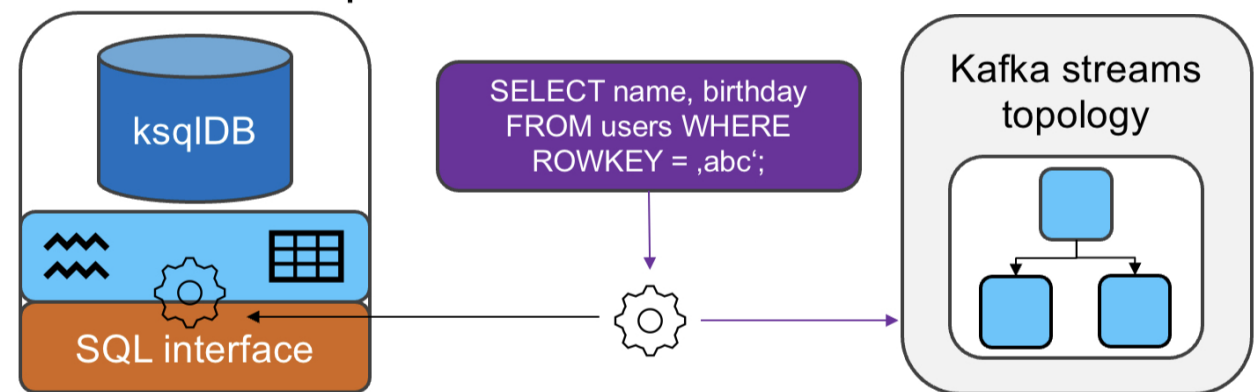
ksqlDB Supports Both Data Integration and Transformation

- ksqlDB extends the classical SQL dialect to support streams
 - Support for both push and pull queries
- Integration of Kafka Connect



→ combines ETL + stream processing

- **ksqlDB Server**
 - conceptually similar to an instance of a Kafka Streams application; like Kafka Streams applications deployed independently of the Kafka cluster; ksqlDB servers can be organized in ksqlDB clusters
- **SQL Engine**
 - Responsible for parsing **SQL** statements and **translating** them into one or more **Kafka Streams topologies** and ultimately running it
- **REST Service**
 - Allows clients to interact with the SQL engine
- **ksqlDB Clients**
 - ksqlDB CLI and ksqlDB UI

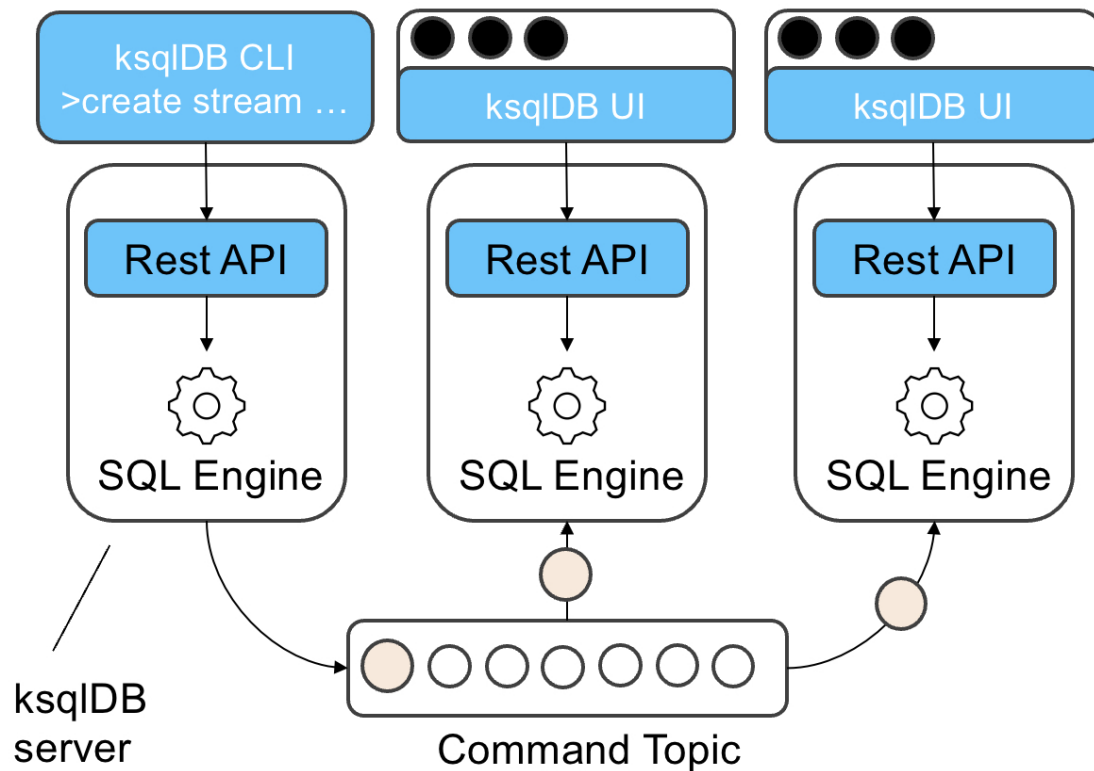


Deployment Modes: Interactive versus Headless

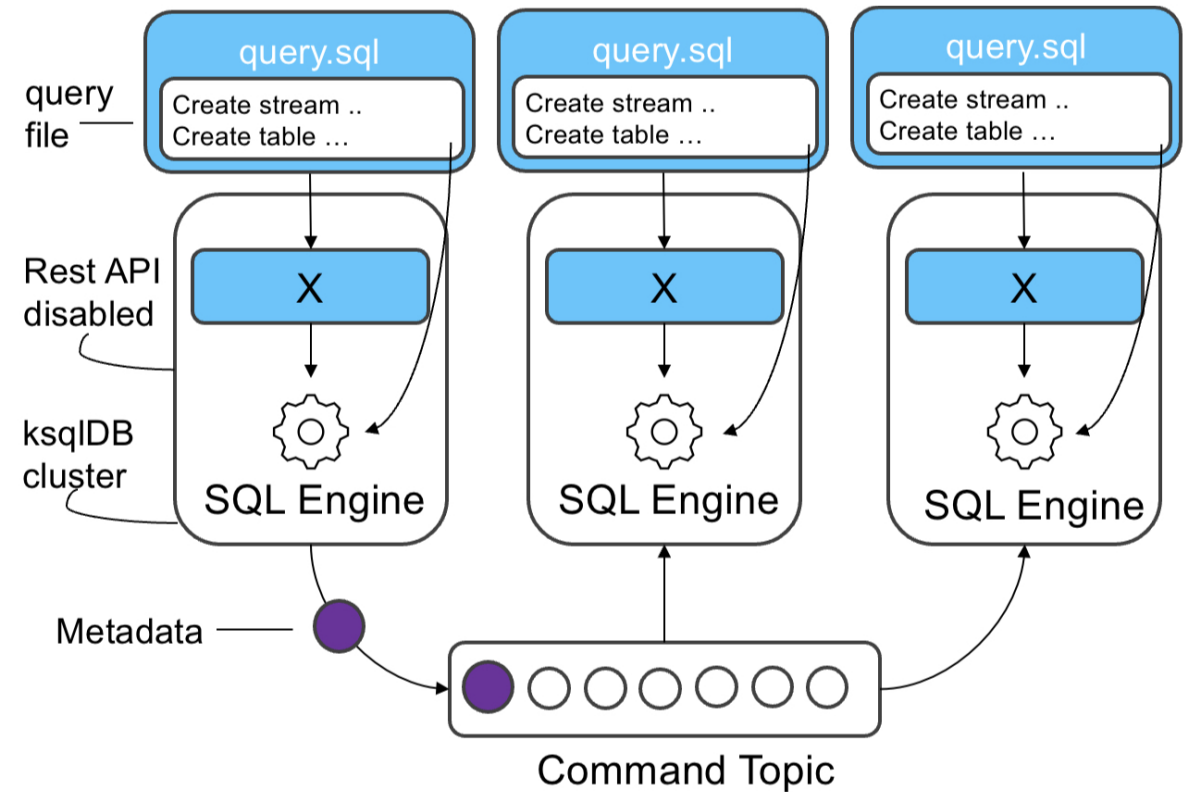
Interactive → exploratory / ad-hoc queries

Headless → production / predefined queries

Interactive Mode



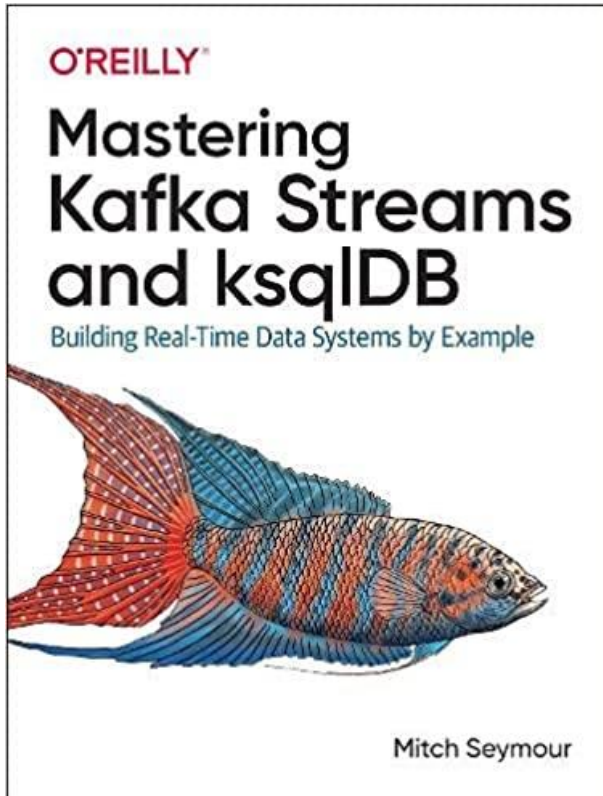
Headless Mode



Kafka Streams vs ksqlDB

- Kafka Streams:
 - Java / code-based
 - Maximum flexibility
 - Complex logic, custom processing
- ksqlDB:
 - SQL-based
 - Fast development
 - Standard transformations (joins, aggregations, filters)

Rule of thumb:→ Use ksqlDB if it fits→ Use Kafka Streams if you need more control



[Get free ebook](#)
via Confluent

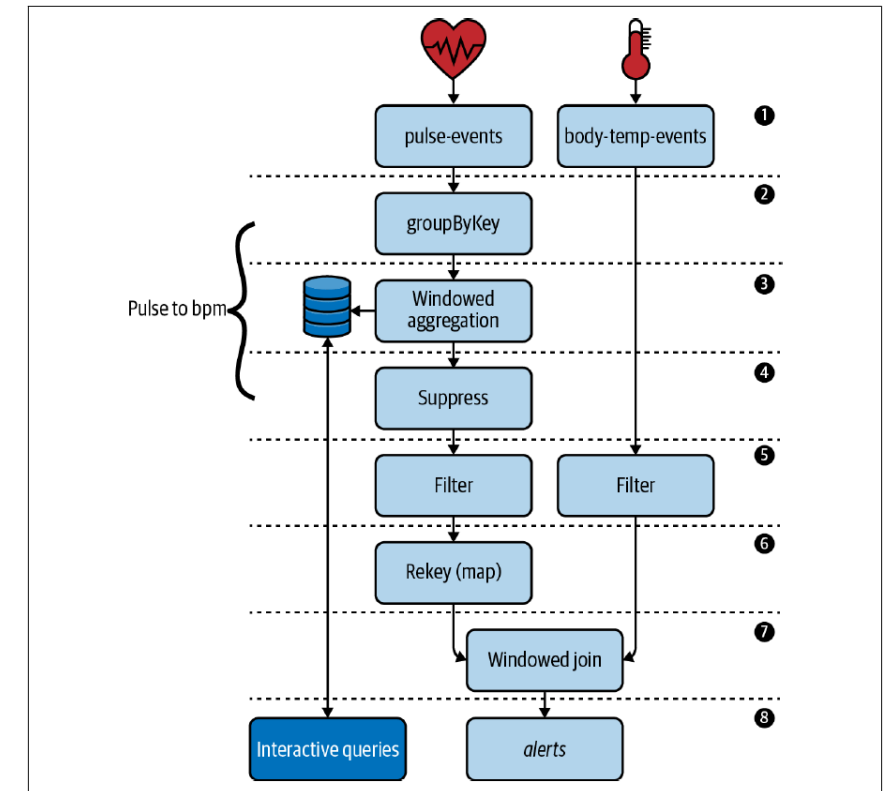
Event Streaming Patterns

<https://developer.confluent.io/patterns>

Kafka Streams Microservices Example

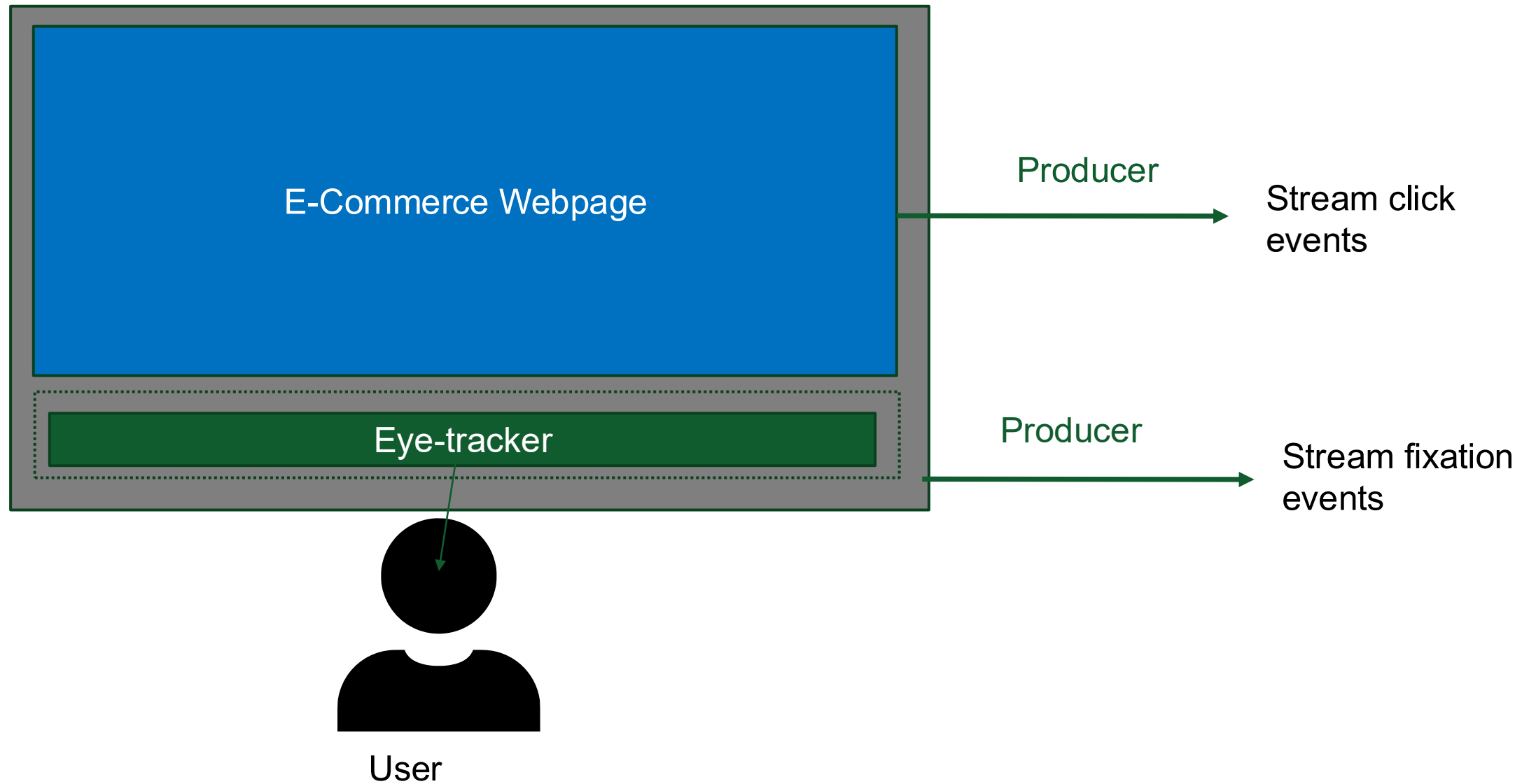
Tutorial Event-driven Microservices with ksqlDB
(checking suspicious credit card transactions)

- For Kafka Streams stateful (part 2):
 - Patient Monitoring App:
 - <https://github.com/scs-edpo/lab14Part1-kafka-streams-patientmonitoring>
 - Chapter 5 in Mastering Kstreams and ksqlDB

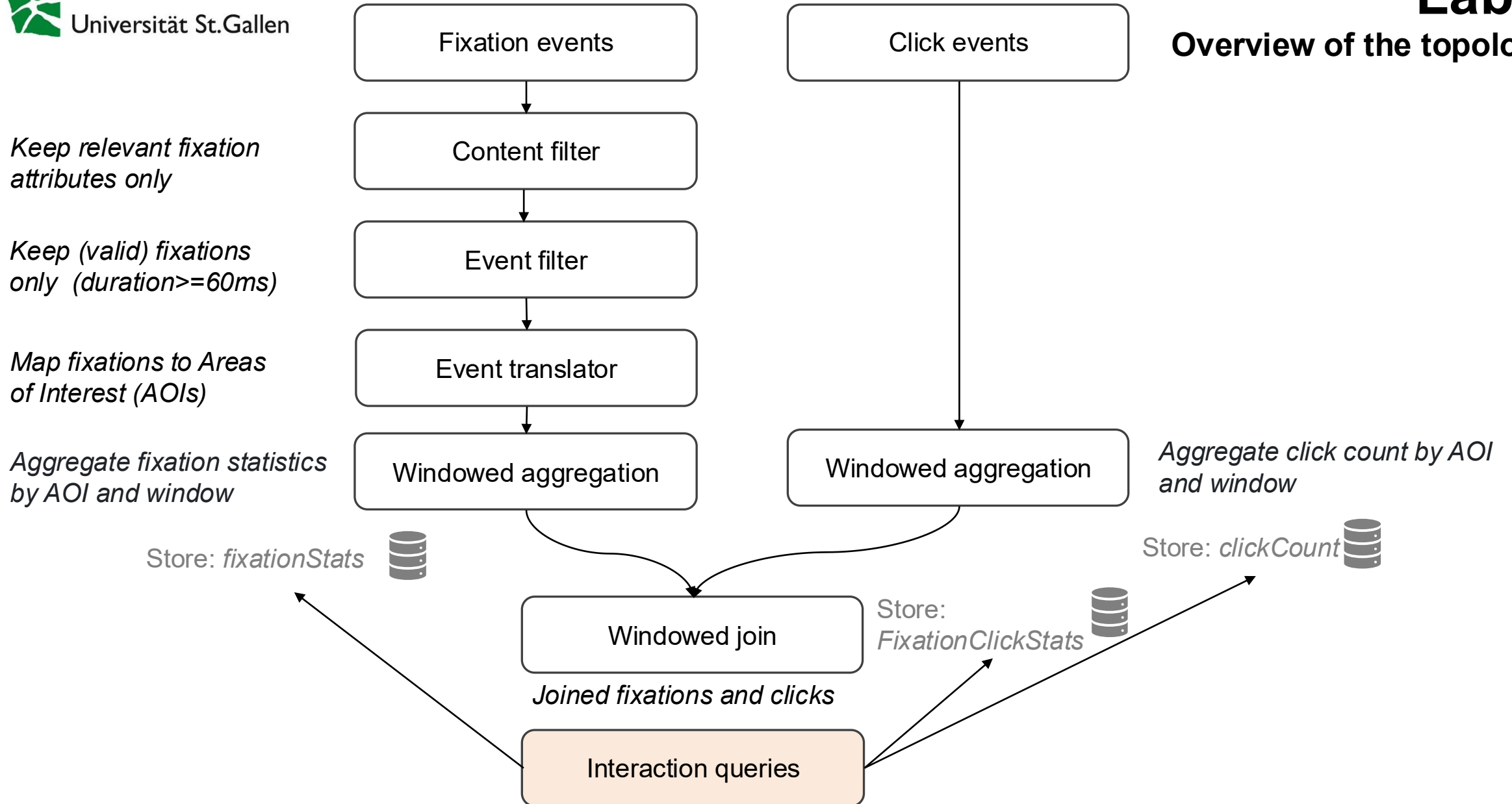


- For Kafka Streams stateless and stateful event processing *including time windowing* in the eye-tracking domain (part 3):
 - Eye-Tracking Events Processing App:
 - <https://github.com/scs-edpo/lab14Part2-kafka-streams-eyeTracking3>

Computer screen



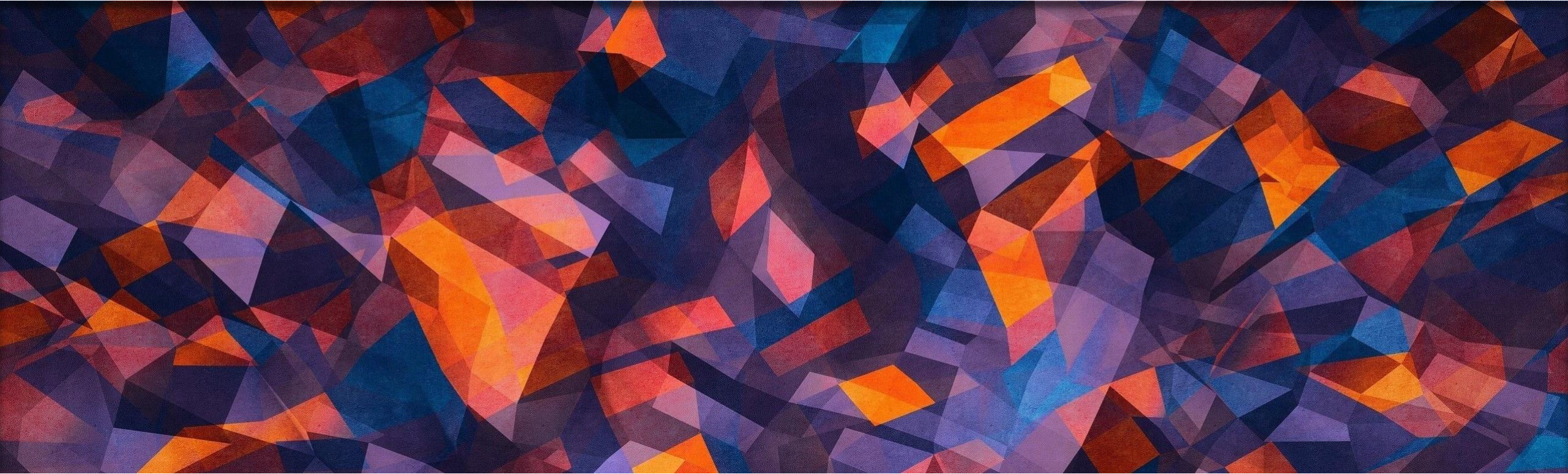
Overview of the topology



- For ksqlDB:
 - Credit Card Transactions App:
 - <https://github.com/scs-edpo/lab15-ksqldb-credittransactions>
 - Tutorial: [Event-driven Microservices with ksqlDB](#)



Universität St.Gallen

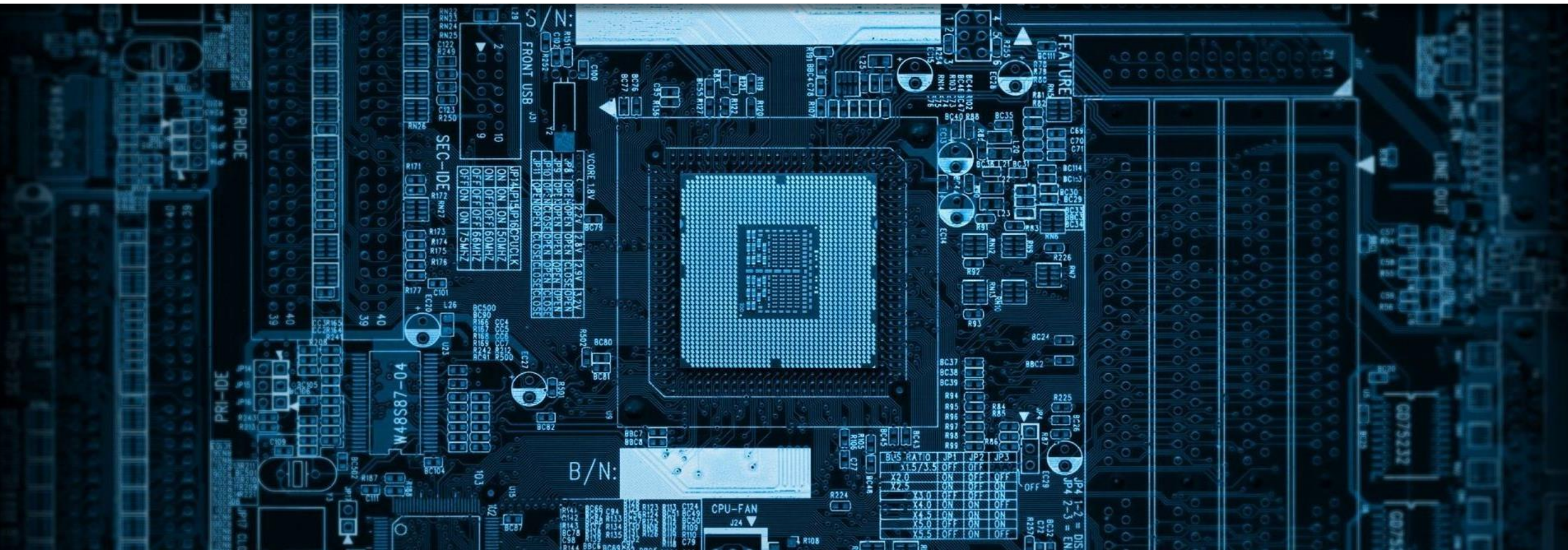


Event-driven and Process-oriented Architectures

Project

Lecture 10

1. Develop one or several stream processing applications
2. The stream processing applications should include both stateless and stateful stream processing
 - At least several stateless operations (Week 8)
 - Usage of both streams and tables (Week 9)
 - Consider data from more than one stream (Week 9)
 - Interactive queries (Week 9)
 - Windowed operations (Week 10)
3. Describe for each developed stream processing application the topology in graphical form
4. Elaborate on the chosen patterns
5. Use Avro schema (registryless or with a schema registry)



Questions?